



Damietta University
Faculty of Computers &
Artificial Intelligence

Graduation Project

2025 – 2026

Automated Vulnerability Scanner
(TRACE)

By

Salem Mohamed Syam

Mostafa Mohamed Radwan

Mohamed Ahmed Abo-Elftoh

Nour El-Islam Hamdy Eid

Youssef Mohamed Rabie

Zeyad Mohamed Nagib

Under the supervision of: Prof. Ahmed Mohammed Rabie

ACKNOWLEDGMENT

We would like to extend our sincere gratitude to Prof. Ahmed Mohammed Rabie for his guidance, support, and mentorship throughout this project. His expertise in cybersecurity and software engineering provided invaluable direction that shaped the development of TRACE.

We also thank each team member for their commitment and collaborative spirit: Mostafa Mohamed El-Sayed Mahmoud Radwan, Ziad Mohamed Nagib El-Badawy, Mohamed Ahmed Abo-Elftouh, Youssef Mohamed Rabie Abdel-Wahab, Nour El-Islam Hamdy Massad Ibrahim Eid, and Salem Mohamed Salem Mohamed Syam. Each member contributed their unique skills to make this project a success.

Special thanks are also due to the Faculty of Computers and Artificial Intelligence at Damietta University for providing the academic environment and resources necessary to complete this work.

This project is the result of a combined effort and dedication to advancing cybersecurity measures and making security testing accessible to organizations of all sizes.

ABSTRACT

In today's rapidly evolving digital landscape, organizations face an ever-increasing volume and sophistication of cybersecurity threats. Traditional vulnerability assessment approaches are often fragmented across multiple tools, require significant manual effort, and lack intelligent analysis capabilities. This creates a critical gap between the growing attack surface and the ability to respond effectively.

TRACE is an automated vulnerability scanning platform designed to address these challenges by providing a unified, intelligent security assessment solution. The platform integrates a modular scan engine comprising 43 specialized Python security scripts covering a comprehensive range of vulnerability categories — from SQL injection and cross-site scripting (XSS) to subdomain takeover, cloud storage exposure, and NoSQL injection. The system supports two scanning modes: Quick Scan for rapid essential checks and Deep Scan for comprehensive coverage.

A key differentiator of TRACE is the integration of Google Gemini AI for automated analysis. Upon scan completion, the AI generates a detailed security assessment including a risk score (0–100), executive summary, key findings with impact analysis, and prioritized remediation recommendations. An interactive AI Chat feature provides context-aware security guidance based on actual scan data.

The platform is built on a modern technology stack: Next.js 15 with React 19 for the frontend dashboard, NestJS for the backend API, PostgreSQL for persistent storage, and Redis with BullMQ for background job orchestration. Real-time scan progress is streamed to clients via Server-Sent Events (SSE), and the system supports multi-tenant organizations with role-based access control (Admin, Editor, Viewer).

TRACE transforms vulnerability scanning from a manual, fragmented process into a continuous, centralized security management platform. The system demonstrated significant efficiency gains, completing comprehensive deep scans in under 3 minutes compared to 20–30 minutes for equivalent manual workflows.

TABLE OF CONTENTS

ACKNOWLEDGMENT.....	I
ABSTRACT.....	II
TABLE OF CONTENTS.....	III
LIST OF FIGURES	VII
LIST OF TABLES	X
LIST OF ABBREVIATIONS.....	XI
CHAPTER 1: INTRODUCTION AND PROJECT OBJECTIVES.....	2
1.1 Project Overview	2
1.2 Problem Statement	3
1.3 Project Objectives	3
1.4 Project Scope.....	4
1.5 Team Vision	5
CHAPTER 2: BACKGROUND AND RELATED WORK.....	7
2.1 Problem Definition.....	7
2.2 OWASP ZAP (Zed Attack Proxy)	8
2.3 Nessus (Tenable).....	9
2.4 Burp Suite (PortSwigger).....	11
2.5 Comparison Summary	13
CHAPTER 3: PROPOSED SOLUTION.....	16
3.1 Development Methodology: Agile Approach	16
3.2 System Requirements.....	18
3.2.1 Functional Requirements.....	18
3.2.2 Non-Functional Requirements.....	20

3.2.3 User Requirements	21
3.3 System Analysis	22
3.3.1 Use-Case Diagram	22
3.3.2 Use-Case Descriptions.....	23
3.3.3 Sequence Diagrams	25
3.3.4 Entity Relationship Diagram (ERD).....	38
3.4 System Architecture	40
3.4.1 Technologies Overview	40
3.4.2 System Architecture Diagram	42
CHAPTER 4: IMPLEMENTATION AND RESULTS	45
4.1 User Interface Design.....	45
4.1.1 Design System	45
4.1.2 Screen-by-Screen Breakdown	47
4.1.2.1 Login Page	47
4.1.2.2 Registration Page.....	48
4.1.2.3 Dashboard Overview	49
4.1.2.4 Asset Management Page.....	50
4.1.2.5 Scan Initiation Page	51
4.1.2.6 Scan Progress — Real-Time Progress Bar	52
4.1.2.7 Scan Results — Findings Table.....	53
4.1.2.8 AI Analysis Page	54
4.1.2.9 AI Chat Interface	55
4.1.2.10 Report Generation Page.....	56
4.1.2.11 Scan Scheduling Page.....	57
4.1.2.12 Notifications Panel	58

4.1.2.13 Organization Settings — Member Management	59
4.1.2.14 Ai Remediation Plan.....	60
4.1.2.15 Ai Insights.....	61
4.1.2.16 Responsive Design	62
4.1.2.17 Dark Mode.....	63
4.1.2.18 Trace Forms	65
4.2 Frontend Architecture	67
4.2.1 Application Structure.....	67
4.2.2 State Management & Security	67
4.3 Backend Architecture	68
4.3.1 Authentication System.....	68
4.3.2 API Endpoints.....	70
4.4 Database Layer	73
4.4.1 Entity Definitions	73
4.4.2 Server Configuration	75
4.5 Scan Engine Layer	75
4.5.1 Script Runner Architecture	76
4.5.2 Scan Orchestrator	77
4.5.3 Scan Phases.....	78
4.5.4 Quick Scan vs Deep Scan.....	79
4.5.5 Aggregator Service	80
4.6 AI Analysis Layer	81
4.6.1 Automated Scan Analysis.....	81
4.6.2 Prompt Engineering.....	82
4.7 Report Generation	83

4.8 Development Tools and Requirements	83
4.9 Testing and Evaluations	84
4.9.1 Testing Methodology.....	84
4.9.2 Test Cases	85
4.9.3 Limitations.....	88
4.9.4 System Evaluation Metric	89
CHAPTER 5: CONCLUSION AND FUTURE WORK	91
5.1 Conclusion.....	91
5.2 Future Work	92
5.2.1 Mobile Application.....	92
5.2.2 CI/CD Integration	92
5.2.3 CVE Database Integration	92
5.2.4 Agent-Based Internal Scanning	93
5.2.5 ML-Based False Positive Reduction	93
5.2.6 Advanced Authentication Handling	93
5.2.7 Compliance Framework Mapping	93
REFERENCES	94
CONCLUSION.....	96

LIST OF FIGURES

Figure 1: OWASP ZAP Logo	8
Figure 2: Nessus Logo	9
Figure 3:Burp Suite Logo	11
Figure 4: TRACE LOGO - Light.....	14
Figure 5: TRACE Logo-Light	14
Figure 6: TRACE Logo-Dark	14
Figure 7: Agile Development Methodology Lifecycle.....	17
Figure 8: Use-Case Diagram.....	22
Figure 9: Sequence Diagram: User Registration	25
Figure 10 : Sequence Diagram: User Registration	25
Figure 11 : Sequence Diagram: User Login:	26
Figure 12 : Sequence Diagram: Organization Switching	27
Figure 13 : Sequence Diagram: User Management	28
Figure 14 : Sequence Diagram: Asset Management.....	29
Figure 15 : Sequence Diagram: Quick Scan	30
Figure 16 : Sequence Diagram: Deep Scan	31
Figure 17 : Sequence Diagram: Scan Scheduling.....	32
Figure 18: Sequence Diagram: Report Generation.....	33
Figure 19 : Sequence Diagram: Notifications.....	34
Figure 20: Sequence Diagram: AI Analysis Generation.....	35
Figure 21: Sequence Diagram: AI Chat.....	36
Figure 22 : Sequence Diagram: Remediation Plan Generation	37
Figure 23: Entity Relationship Diagram (ERD)	39
Figure 24 : System Architecture Diagram	42
Figure 25 : TRACE Design System.....	46
Figure 26 : Login Page.....	47
Figure 27: Registration Page.....	48
Figure 28 : Dashboard Overview	49
Figure 29 : Asset Management Page	50

Figure 30 : Scan Initiation Page.....	51
Figure 31 : Real-Time Progress Bar	52
Figure 32 : Scan Results — Findings Table	53
Figure 33: AI Analysis Page	54
Figure 34: AI Chat Interface	55
Figure 35: Report Generation Page.....	56
Figure 36 : Scan Scheduling Page	57
Figure 37: Notifications Panel	58
Figure 38 : Organization Settings — Member Management.....	59
Figure 39: Ai Remediation Plan	60
Figure 40: Ai Insights	61
Figure 41: Assets Mobile Screen	62
Figure 42: Scan Progress Mobile Screen.....	62
Figure 43 : Dashboard Mobile Screen	62
Figure 44: Dark Landing Page.....	63
Figure 45 : Dark Dashboard.....	64
Figure 46 : Dark Findings Page	64
Figure 47: Initiate Scan Form	65
Figure 48 : Add Schedule Form.....	65
Figure 49: Bulk Asset Register Form	65
Figure 50: Register Asset Form	65
Figure 51 : Generate Report Form	65
Figure 52 : Inviter Member Form	65
Figure 53 : Invite Team Member Form	66
Figure 54: Generate Report Form	66
Figure 55: JWT Authentication Strategy (Code)	68
Figure 56 : JWT Auth Guard (Code)	69
Figure 57: User Entity — TypeORM Definition (Code).....	73
Figure 58 : Scan Entity — TypeORM Definition (Code).....	74
Figure 59 : ScriptRunnerService — Python Process Spawning (Code)	76
Figure 60 : ScanOrchestratorService — Phase-Based Execution (Code)	77

Figure 61 : AggregatorService — Finding Normalization (Code)	80
Figure 62 : AI Analysis Service — Event-Driven Trigger (Code).....	81
Figure 63 : Prompt Builder Service — Gemini Prompt Construction (Code).....	82

LIST OF TABLES

Table 1: Comparison of TRACE with Existing Security Tools	13
Table 2: Functional Requirements	19
Table 3: Non-Functional Requirements.....	21
Table 4: Use-Case Descriptions.....	24
Table 5: API Endpoints.....	72
Table 6 : Server Configuration	75
Table 7 : Scan Phases and Scripts.....	78
Table 8 : Quick Scan vs Deep Scan Comparison	79
Table 9 : Report Generation Formats.....	83
Table 10 : Development Tools and Technologies	83
Table 11 : Test Cases and Results.....	87

LIST OF ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
AI	Artificial Intelligence
CIDR	Classless Inter-Domain Routing
CI/CD	Continuous Integration / Continuous Deployment
CORS	Cross-Origin Resource Sharing
CRLF	Carriage Return Line Feed
CSRF	Cross-Site Request Forgery
CSS	Cascading Style Sheets
CVE	Common Vulnerabilities and Exposures
CVSS	Common Vulnerability Scoring System
DNS	Domain Name System
DTO	Data Transfer Object
ERD	Entity Relationship Diagram
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IDOR	Insecure Direct Object Reference
IoT	Internet of Things
JSON	JavaScript Object Notation
JWT	JSON Web Token
LDAP	Lightweight Directory Access Protocol
LFI	Local File Inclusion
ML	Machine Learning
NoSQL	Not Only SQL
ORM	Object-Relational Mapping

OWASP	Open Web Application Security Project
PDF	Portable Document Format
RBAC	Role-Based Access Control
REST	Representational State Transfer
SQL	Structured Query Language
SQLi	SQL Injection
SSE	Server-Sent Events
SSRF	Server-Side Request Forgery
SSL	Secure Sockets Layer
SSTI	Server-Side Template Injection
TLS	Transport Layer Security
TOC	Table of Contents
UI	User Interface
UML	Unified Modeling Language
URL	Uniform Resource Locator
UX	User Experience
WAF	Web Application Firewall
XSS	Cross-Site Scripting
XXE	XML External Entity

Chapter 1

Introduction and Project Objectives

CHAPTER 1: INTRODUCTION AND PROJECT OBJECTIVES

1.1 Project Overview

The digital transformation era has brought unprecedented connectivity and convenience to organizations worldwide. However, this expansion of digital infrastructure has simultaneously created a vast and ever-growing attack surface for cyber threats. According to recent cybersecurity reports, the frequency and sophistication of cyberattacks continue to rise year over year, affecting businesses of all sizes across every industry. Web applications and network infrastructure remain among the most targeted vectors for malicious actors seeking to exploit vulnerabilities for data theft, service disruption, and unauthorized access.

Vulnerability scanning — the process of systematically identifying security weaknesses in computer systems, networks, and applications — has become a critical component of any organization's security posture. However, the current landscape of vulnerability scanning tools presents significant challenges for security teams and system administrators. Most existing tools focus on narrow aspects of security testing, forcing practitioners to use multiple disconnected tools to achieve comprehensive coverage. This fragmentation leads to inefficient workflows, inconsistent reporting formats, and a lack of unified visibility into an organization's overall security health.

Furthermore, interpreting raw scan results requires significant domain expertise. Security teams must manually analyze findings, prioritize remediation efforts, and communicate technical risks to non-technical stakeholders. This manual analysis process is time-consuming, error-prone, and creates bottlenecks that delay critical security improvements.

TRACE was conceived to address these fundamental challenges. It is an automated vulnerability scanning platform that unifies network and web application security testing into a single, intelligent system. By combining a comprehensive modular scan engine with artificial intelligence-powered analysis, TRACE transforms the vulnerability assessment process from a fragmented, manual workflow into an automated, continuous security management platform.

1.2 Problem Statement

Organizations today face several critical problems when conducting vulnerability assessments:

- **Tool Fragmentation:** Security teams must use multiple disconnected tools — one for port scanning (e.g., Nmap), another for web application testing (e.g., OWASP ZAP), and separate tools for specific vulnerability categories. Each tool has its own interface, configuration, and output format, creating a disjointed workflow.
- **Manual Analysis Overhead:** Raw scan results typically consist of technical data that requires expert interpretation. Security professionals spend significant time manually reviewing findings, cross-referencing with known vulnerability databases, and formulating remediation recommendations.
- **Lack of Unified Reporting:** Different tools produce outputs in incompatible formats, making it difficult to create consolidated security reports for management and compliance purposes.
- **Steep Learning Curves:** Many existing security tools require extensive command-line expertise and deep security knowledge, creating barriers for organizations without dedicated security teams.
- **No Continuous Monitoring:** Most tools provide point-in-time assessments without built-in scheduling or trend analysis capabilities, leaving gaps between assessments where new vulnerabilities may go undetected.

These challenges result in delayed vulnerability detection, inconsistent security coverage, and increased risk exposure for organizations that cannot afford dedicated security teams or expensive commercial solutions.

1.3 Project Objectives

TRACE aims to achieve the following objectives:

1. Develop a modular, extensible scan engine that covers both network infrastructure and web application vulnerabilities through 43+ specialized Python security scripts.

2. Integrate AI-powered analysis using Google Gemini to automatically generate risk scores, executive summaries, key findings with impact assessments, and prioritized remediation recommendations.
3. Provide real-time scan monitoring through Server-Sent Events (SSE) for live progress tracking with phase-level status updates.
4. Build a multi-tenant platform with Role-Based Access Control (RBAC) supporting Admin, Editor, and Viewer roles for team-based security management.
5. Implement automated report generation in multiple formats (PDF, HTML, JSON) for flexible security documentation and compliance reporting.
6. Enable recurring scan scheduling (Daily, Weekly, Monthly) for continuous security monitoring without manual intervention.
7. Create an interactive AI Chat assistant that provides context-aware security guidance based on actual scan data.

1.4 Project Scope

TRACE focuses on the following areas within its defined scope:

In Scope:

- Web application vulnerability scanning (SQL injection, XSS, CSRF, SSRF, and 30+ additional vulnerability categories).
- Network infrastructure scanning (port scanning, service fingerprinting, SSL/TLS analysis, DNS reconnaissance).
- AI-powered automated analysis and remediation recommendations.
- Multi-format report generation (PDF, HTML, JSON).
- Multi-tenant organization management with role-based access control.
- Scan scheduling and continuous monitoring.
- Asset management for domains, IP addresses, URLs, and CIDR ranges.

Out of Scope:

- Mobile application security testing (iOS/Android).
- IoT device scanning.
- Source code analysis (static application security testing — SAST).
- Physical security assessments.
- Social engineering testing.

1.5 Team Vision

Our team is committed to democratizing cybersecurity by making professional-grade vulnerability assessment accessible to organizations of all sizes. We believe that every organization, regardless of its technical resources, should have the ability to understand and manage its security posture effectively.

With a focus on automation, intelligence, and user-friendly design, we aim to make TRACE a platform that bridges the gap between complex security testing tools and the practical needs of modern development teams. Our vision extends beyond simple scanning — we envision a platform where AI-assisted security insights empower teams to make informed decisions about their security investments, prioritize remediation efforts based on actual risk, and maintain continuous visibility into their evolving threat landscape.

Through collaborative development and a commitment to open, extensible architecture, our team aspires to build a platform that can evolve alongside the ever-changing cybersecurity landscape, incorporating new vulnerability checks, enhanced AI models, and expanded integration capabilities as the field progresses.

Chapter 2

Background and Related Work

CHAPTER 2: BACKGROUND AND RELATED WORK

2.1 Problem Definition

The cybersecurity landscape has undergone a dramatic transformation in recent years. As organizations increasingly rely on web applications and networked systems for their core operations, the attack surface available to malicious actors has expanded significantly. The Open Web Application Security Project (OWASP) regularly publishes updated lists of the most critical web application security risks, highlighting vulnerabilities such as injection attacks, broken authentication, and security misconfiguration that continue to affect millions of web applications worldwide [1].

Conducting thorough vulnerability assessments remains a complex and resource-intensive process. Security professionals typically rely on a combination of specialized tools, each designed to address specific aspects of security testing. A typical assessment workflow might involve using Nmap for network discovery and port scanning [2], OWASP ZAP for web application vulnerability detection [3], and separate tools for SSL/TLS analysis, DNS reconnaissance, and specific vulnerability categories such as SQL injection or cross-site scripting.

This fragmented approach presents several challenges. First, each tool requires separate installation, configuration, and learning investment. Second, the outputs from different tools are typically incompatible, requiring manual effort to consolidate findings into a coherent security picture. Third, interpreting raw scan results and formulating actionable remediation plans requires significant domain expertise that many organizations lack.

Furthermore, existing tools generally provide point-in-time assessments without built-in capabilities for continuous monitoring, trend analysis, or intelligent prioritization of findings. As new vulnerabilities are discovered daily and applications are updated frequently, this static approach leaves dangerous gaps in security coverage.

TRACE addresses these challenges by providing a unified platform that combines comprehensive scanning capabilities with AI-powered analysis, automated reporting, and continuous monitoring — all accessible through an intuitive web dashboard.

2.2 OWASP ZAP (Zed Attack Proxy)



Figure 1: OWASP ZAP Logo

OWASP ZAP [3] is one of the world's most widely used open-source web application security scanners. Originally developed by the OWASP Foundation, ZAP is designed to help security professionals and developers find vulnerabilities in web applications during the development and testing phases. It functions as an intercepting proxy, sitting between the tester's browser and the web application to inspect and manipulate HTTP traffic.

Key Features:

- **Automated Scanner:** Performs automated crawling and active scanning of web applications to detect common vulnerabilities including SQL injection, XSS, and broken authentication.
- **Intercepting Proxy:** Allows manual inspection and modification of HTTP requests and responses between the browser and the target application.
- **Passive Scanning:** Continuously analyzes proxied traffic for security issues without sending additional requests to the target.
- **Extensive API:** Provides a REST API for integration with CI/CD pipelines and other tools.
- **Add-on Marketplace:** Supports community-contributed extensions for additional functionality.

Differences with TRACE:

- TRACE provides both network infrastructure scanning (port scanning, service fingerprinting, DNS recon) and web application testing in a single platform, whereas ZAP focuses exclusively on web application vulnerabilities.

- TRACE integrates Google Gemini AI to automatically analyze scan results, generate risk scores, and provide prioritized remediation recommendations. ZAP provides raw findings without AI-powered analysis.
- TRACE offers a modern web dashboard with role-based access control for multi-team collaboration, while ZAP is primarily a desktop application designed for individual use.
- TRACE includes built-in scan scheduling, automated report generation in multiple formats (PDF, HTML, JSON), and an interactive AI Chat assistant — features not available in ZAP.
- ZAP excels in manual penetration testing scenarios with its proxy capabilities, whereas TRACE is optimized for automated, continuous security monitoring.

2.3 Nessus (Tenable)



Figure 2: Nessus Logo

Nessus, developed by Tenable Inc., is one of the most popular commercial vulnerability scanners in the cybersecurity industry. First released in 1998, Nessus has evolved into a comprehensive vulnerability assessment solution used by security professionals worldwide. The tool is available in both a free community edition (Nessus Essentials) with limited capabilities and a paid professional edition (Nessus Professional) for enterprise use.

Key Features:

- **Extensive Plugin Library:** Contains over 100,000 plugins for detecting known vulnerabilities across a wide range of operating systems, applications, and network devices.

- **Compliance Auditing:** Supports compliance checking against standards such as PCI DSS, HIPAA, and CIS benchmarks.
- **Credentialed Scanning:** Can authenticate to target systems for deeper internal vulnerability assessment.
- **Agent-Based Scanning:** Deploys lightweight agents on target systems for continuous monitoring.
- **Reporting:** Generates detailed vulnerability reports with severity ratings and remediation guidance.

Differences with TRACE:

- Nessus is a commercial product with significant licensing costs, making it inaccessible to many small and medium-sized organizations. TRACE is designed to be accessible without costly licensing fees.
- TRACE integrates AI-powered analysis using Google Gemini for intelligent risk assessment and natural-language recommendations. Nessus relies on pre-written plugin descriptions without AI-driven contextual analysis.
- TRACE provides a modern, responsive web dashboard built with Next.js, while Nessus uses a more traditional web interface.
- TRACE focuses on web application and network scanning with 43+ specialized scripts. Nessus covers a broader scope including operating system vulnerabilities and compliance auditing, but with less depth in web application testing.
- TRACE offers an interactive AI Chat assistant for conversational security guidance, a unique feature not found in Nessus.

2.4 Burp Suite (PortSwigger)



Figure 3:Burp Suite Logo

Burp Suite, developed by PortSwigger, is a leading integrated platform for web application security testing. It is widely used by professional penetration testers and is considered an industry-standard tool for manual and automated web vulnerability assessment. Burp Suite is available in a free Community Edition with limited features and a paid Professional Edition with full scanning capabilities.

Key Features:

- Intercepting Proxy: Comprehensive HTTP/HTTPS interception and modification capabilities with advanced request/response analysis.
- Automated Scanner: Professional edition includes an advanced automated vulnerability scanner with sophisticated crawling and detection capabilities.
- Intruder: Powerful tool for automated customized attacks, useful for fuzzing and brute-force testing.
- Repeater: Allows manual manipulation and resending of individual requests for targeted testing.
- Extensibility: Supports custom extensions through the BApp Store and a powerful API for plugin development.

Differences with TRACE:

- Burp Suite Professional is a premium commercial tool with significant annual licensing costs. TRACE provides comprehensive scanning capabilities without commercial licensing requirements.
- TRACE combines web application and network infrastructure testing in a unified platform, while Burp Suite focuses exclusively on web application security.
- TRACE provides AI-powered analysis with Google Gemini for automated risk assessment and remediation recommendations, while Burp Suite relies on its scanning engine's rule-based detection without AI analysis.
- TRACE is designed for continuous, automated scanning with scheduling and real-time progress monitoring. Burp Suite is primarily designed for interactive, manual penetration testing sessions.
- TRACE includes multi-tenant organization management with role-based access control, enabling team-based security workflows. Burp Suite is designed for individual security professionals.

2.5 Comparison Summary

Feature	TRACE	OWASP ZAP	Nessus	Burp Suite
AI-Powered Analysis	✓ (Google Gemini)	✗	✗	✗
AI Chat Assistant	✓	✗	✗	✗
Web App Scanning	✓ (43+ scripts)	✓	✓ (plugins)	✓
Network Scanning	✓ (Nmap integration)	✗	✓	✗
Real-Time Progress	✓ (SSE)	✗	✗	✗
Multi-Tenant / RBAC	✓ (Admin/Editor/Viewer)	✗	✗	✗
Auto Report Generation	✓ (PDF/HTML/JSON)	Limited	✓ (PDF)	✓ (HTML)
Scan Scheduling	✓ (Daily/Weekly/Monthly)	✗	✓	✗
Web Dashboard	✓ (Next.js)	✗ (Desktop)	✓ (Web UI)	✗ (Desktop)
Open Source	✓	✓	✗ (Freemium)	✗ (Freemium)
Interactive Proxy	✗	✓	✗	✓
Compliance Auditing	Planned	✗	✓	✗

Table 1: Comparison of TRACE with Existing Security Tools

As shown in Table 1, TRACE distinguishes itself through its unique combination of AI-powered analysis, unified web and network scanning, real-time monitoring, and multi-tenant collaboration features. While existing tools excel in specific areas — ZAP in interactive proxy testing, Nessus in compliance auditing, and Burp Suite in manual penetration testing — TRACE provides a comprehensive, automated platform designed for continuous security monitoring that is accessible to teams without specialized security expertise

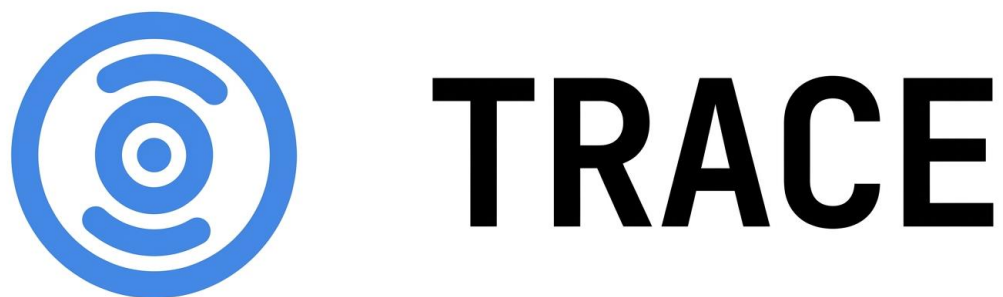


Figure 5: TRACE Logo-Light



Figure 6: TRACE Logo-Dark

Chapter 3

Proposed Solution

CHAPTER 3: PROPOSED SOLUTION

3.1 Development Methodology: Agile Approach

The development of TRACE followed the Agile software development methodology. Agile is a project management and software development approach that emphasizes iterative development, collaboration, and the ability to adapt to changing requirements. Unlike traditional waterfall methodologies which follow a rigid, sequential process, Agile embraces flexibility and continuous improvement through short development cycles called sprints.

The Agile methodology was chosen for TRACE for several key reasons:

- **Incremental Delivery:** The modular nature of TRACE — with its separate scan engine scripts, backend services, and frontend components — naturally lends itself to incremental feature delivery across sprints.
- **Continuous Testing:** Each vulnerability scanning script could be developed, tested, and integrated independently, allowing for continuous validation of scan accuracy.
- **Flexibility:** As new vulnerability categories were identified during development, they could be added to the scan engine without disrupting ongoing work on other components.
- **Collaboration:** The team of six members could work in parallel on different components (frontend, backend, scan scripts) with regular integration points.
- **Quality Focus:** Regular sprint reviews ensured that each component met quality standards before moving to the next development phase.



Figure 7: Agile Development Methodology Lifecycle

The development was organized into the following major sprints:

- Sprint 1–2: Core infrastructure — NestJS backend setup, PostgreSQL schema design, authentication system (JWT), and basic Next.js frontend scaffolding.
- Sprint 3–4: Scan engine foundation — ScriptRunnerService, first batch of Python scanning scripts (SQL injection, XSS, CSRF), BullMQ job queue integration.
- Sprint 5–6: Extended scanning — remaining 30+ vulnerability scripts, scan orchestration with phase-based execution, aggregator service for finding normalization.
- Sprint 7–8: AI integration — Google Gemini integration, prompt engineering, automated analysis on scan completion, AI Chat feature.
- Sprint 9–10: Dashboard and UX — complete frontend implementation, real-time SSE progress, report generation, scan scheduling, notifications, organization management.
- Sprint 11–12: Testing, optimization, and documentation — end-to-end testing, performance optimization, Docker containerization, and documentation.

3.2 System Requirements

3.2.1 Functional Requirements

Functional requirements define the specific behaviors, actions, and capabilities that the system must provide to its users. Table 2 outlines the identified functional requirements for TRACE.

No.	Requirement	Description
1	User Authentication	The system shall provide secure user registration and login using email and password. JWT-based tokens (access + refresh) shall be used for session management.
2	Asset Management	Users shall be able to add, view, and delete digital assets including domains, IP addresses, URLs, and CIDR ranges for scanning.
3	Scan Execution	Users shall be able to initiate vulnerability scans in two modes: Quick Scan (essential checks, ~10 scripts) and Deep Scan (comprehensive, 43+ scripts).
4	Real-Time Progress	The system shall display real-time scan progress via SSE, showing completion percentage and phase-level status messages.
5	Scan Cancellation	Users shall be able to cancel running scans at any time. Cancellation shall immediately terminate all active scanning processes.
6	Vulnerability Reporting	The system shall display detected vulnerabilities with severity levels (Critical, High, Medium, Low),

		categories, evidence, and remediation guidance.
7	AI Analysis	Upon scan completion, the system shall automatically generate an AI-powered analysis including risk score (0–100), executive summary, key findings, and prioritized recommendations.
8	AI Chat	The system shall provide an interactive AI assistant capable of answering security questions with context from actual scan data.
9	Report Generation	Users shall be able to generate downloadable reports in PDF, HTML, and JSON formats from completed scans.
10	Scan Scheduling	Users shall be able to configure recurring scans with configurable frequency (Daily, Weekly, Monthly), day, and time.
11	Notifications	The system shall alert users of scan completion, failures, AI analysis readiness, and critical vulnerability discoveries.
12	Organization Management	Administrators shall be able to manage team members with role-based access: Admin (full access), Editor (manage assets/scans), Viewer (read-only).

Table 2: Functional Requirements

3.2.2 Non-Functional Requirements

Non-functional requirements define the quality attributes and operational constraints of the system. Table 3 presents the non-functional requirements for TRACE.

No.	Requirement	Description
1	Performance	The system shall respond to user actions within 2 seconds. Quick Scans shall complete within 2 minutes. Deep Scans shall complete within 5 minutes for typical targets.
2	Scalability	The system shall support concurrent scans through BullMQ job queue management, scaling to handle multiple organizations and users simultaneously.
3	Availability	The system shall maintain 99% uptime with Docker-based deployment for easy recovery and horizontal scaling.
4	Reliability	Individual script failures shall not crash the entire scan. The system shall use Promise.allSettled for fault isolation across parallel script execution.
5	Security	User passwords shall be hashed using bcrypt. All API endpoints shall be protected by JWT authentication. Input shall be sanitized to prevent command injection.
6	Maintainability	The system shall follow modular architecture. Each scanning script shall be independent and follow a

		strict Runner Contract for easy addition of new checks.
7	Usability	The dashboard shall provide an intuitive interface accessible to users without specialized security expertise, with clear severity indicators and actionable recommendations.

Table 3: Non-Functional Requirements

3.2.3 User Requirements

TRACE serves three distinct user roles, each with specific requirements:

Admin Users:

- Full access to all system features including organization settings and member management.
- Ability to invite team members and assign roles (Editor, Viewer).
- Access to all scan results, reports, and AI analyses across the organization.
- Ability to configure scan schedules and notification preferences.

Editor Users:

- Ability to add, modify, and delete digital assets.
- Ability to initiate, cancel, and monitor scans.
- Access to scan results, AI analysis, and report generation.
- Cannot modify organization settings or manage team members.

Viewer Users:

- Read-only access to scan results, vulnerability findings, and AI analyses.
- Ability to view and download generated reports.
- Cannot initiate scans, modify assets, or change system configurations.

3.3 System Analysis

3.3.1 Use-Case Diagram

Figure 8 shows the use-case diagram for TRACE, illustrating the primary interactions between the three user roles (Admin, Editor, Viewer) and the system's core features. The diagram captures the functional boundaries and permission levels across asset management, scan execution, AI analysis, reporting, and organization management.

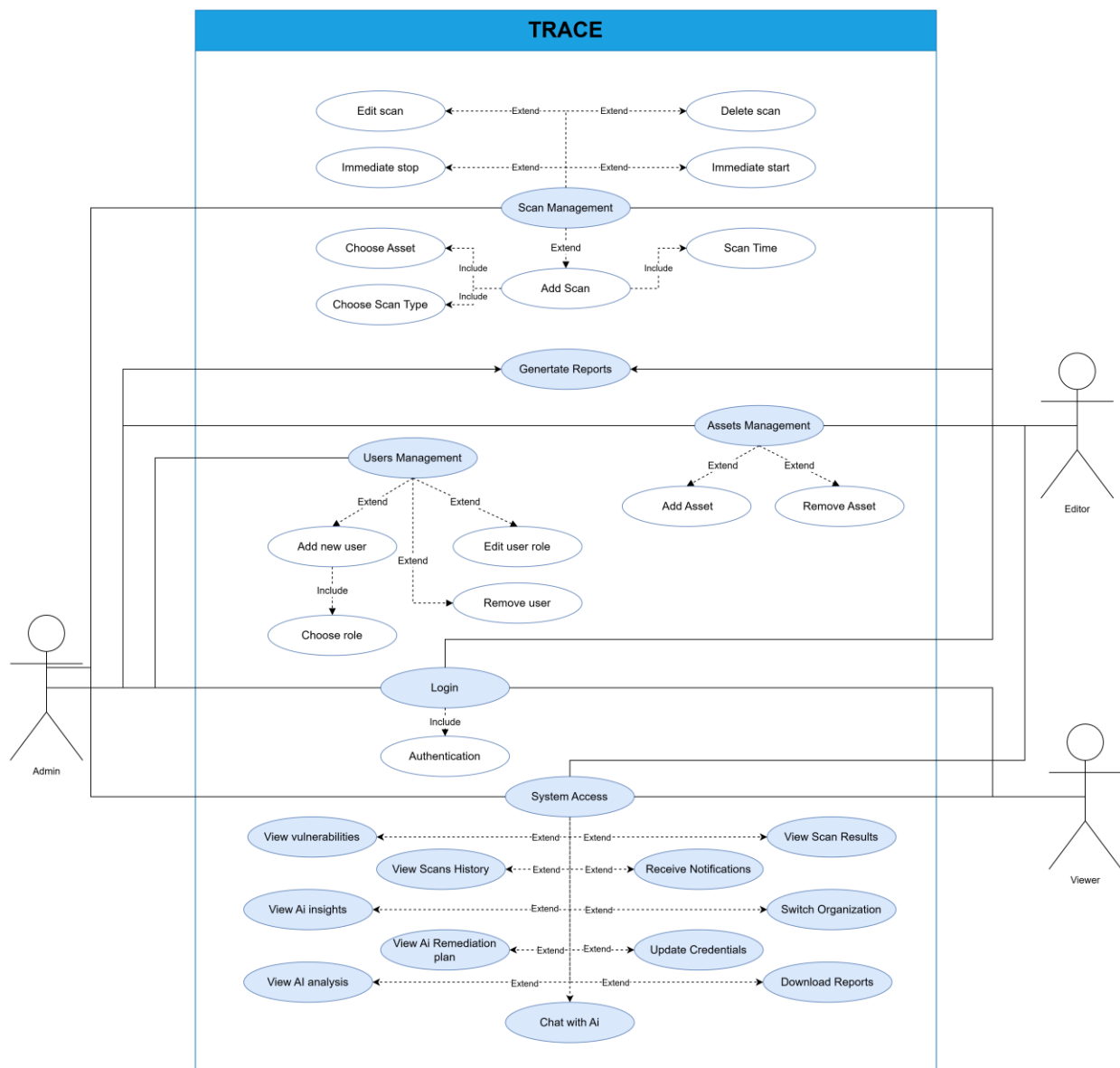


Figure 8: Use-Case Diagram

3.3.2 Use-Case Descriptions

Table 4 provides detailed descriptions of the primary use cases identified in the system.

ID	Use Case	Actors	Description
UC-01	User Registration	Any user	User creates account with name, email, and password. System creates user, default organization, and returns JWT tokens.
UC-02	User Login	Any user	User authenticates with email and password. System validates credentials and returns access/refresh JWT tokens.
UC-03	Add Asset	Admin, Editor	User adds a digital asset (domain, IP, URL, or CIDR) to the organization. System validates the format and stores the asset.
UC-04	Initiate Scan	Admin, Editor	User selects an asset and scan type (Quick/Deep). System creates a scan job in BullMQ and begins execution.
UC-05	Monitor Scan Progress	Admin, Editor, Viewer	User views real-time scan progress via SSE stream showing percentage and phase status messages.
UC-06	Cancel Scan	Admin, Editor	User cancels a running scan. System aborts all active script processes

			and updates scan status to CANCELLED.
UC-07	View Results	Admin, Editor, Viewer	User views detected vulnerabilities in a sortable, filterable table with severity, category, and evidence.
UC-08	View AI Analysis	Admin, Editor, Viewer	User views the AI-generated analysis including risk score, executive summary, and recommendations.
UC-09	Chat with AI	Admin, Editor, Viewer	User asks security questions in a chat interface. AI responds with context from actual scan data.
UC-10	Generate Report	Admin, Editor	User generates a downloadable report (PDF/HTML/JSON) from a completed scan.
UC-11	Schedule Scan	Admin, Editor	User configures a recurring scan with frequency, day, and time parameters.
UC-12	Manage Members	Admin	Admin invites or removes team members and assigns roles (Admin, Editor, Viewer).

Table 4: Use-Case Descriptions

3.3.3 Sequence Diagrams

The following sequence diagrams illustrate the key operational flows within TRACE.

Figure 10 shows the user registration sequence, detailing the flow from form submission through validation, password hashing, organization creation, and JWT token generation.

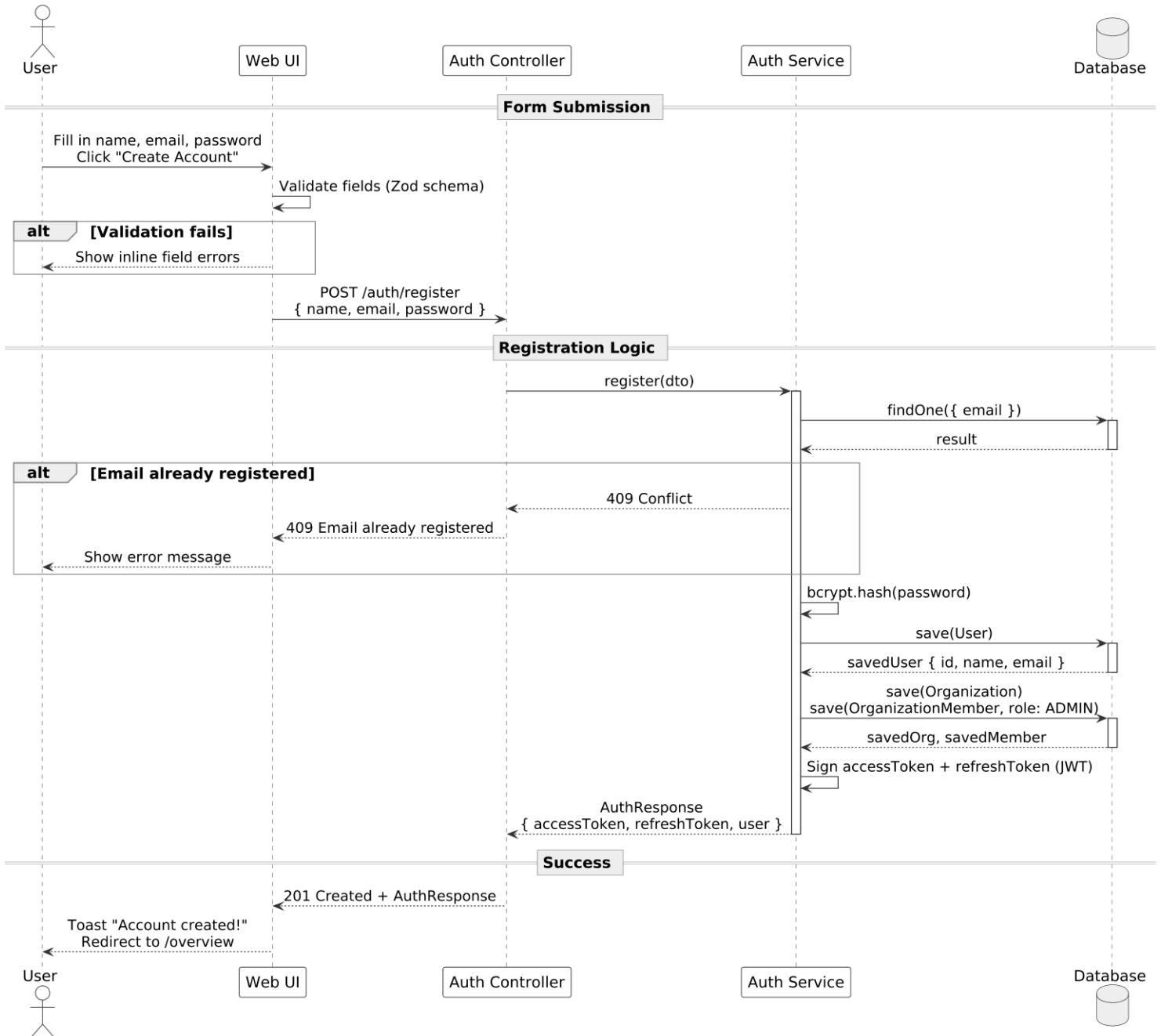


Figure 10 : Sequence Diagram: User Registration

Figure 11 Shows the login flow covering credential validation, bcrypt password comparison, organization membership lookup, and JWT token issuance.

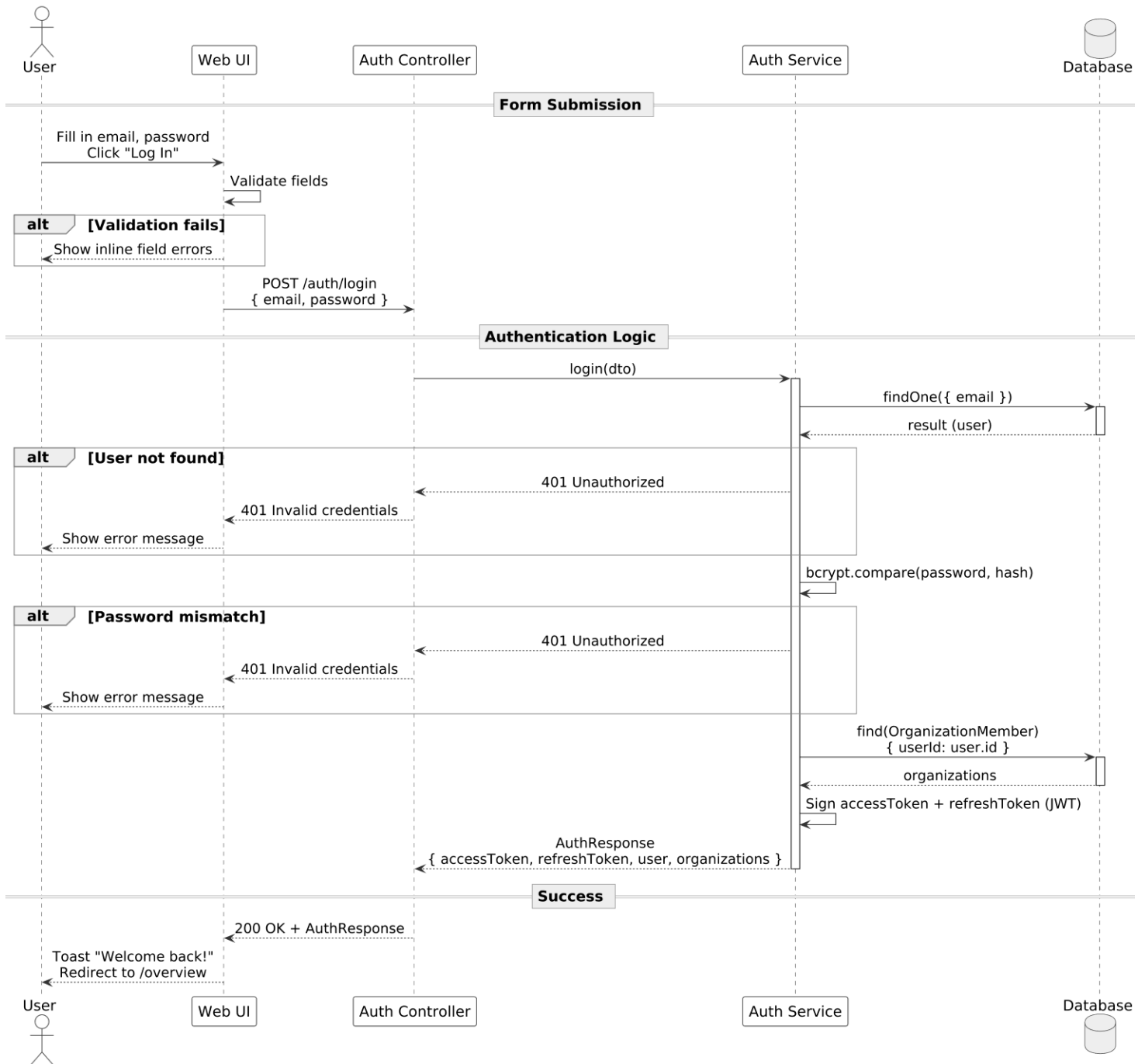


Figure 11 : Sequence Diagram: User Login:

Figure 12 Shows how a user switches between organizations — a client-side only operation that updates `localStorage`, clears all cached data, and redirects to the dashboard. All subsequent API requests are automatically scoped to the new organization via the `x-org-id` header.

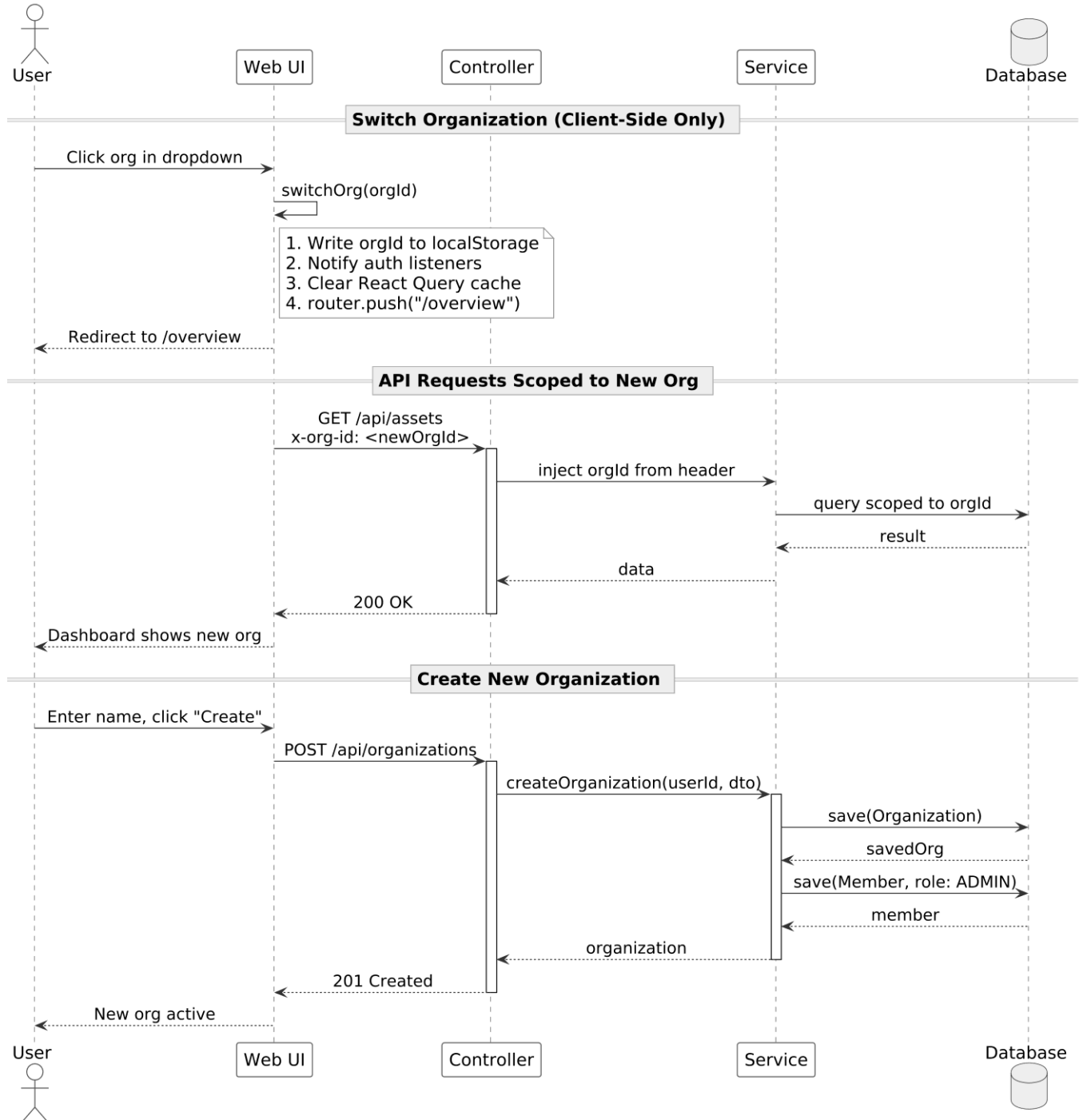


Figure 12 : Sequence Diagram: Organization Switching

Figure 13 Shows how an Admin invites members by email, updates their roles, and removes them from the organization.

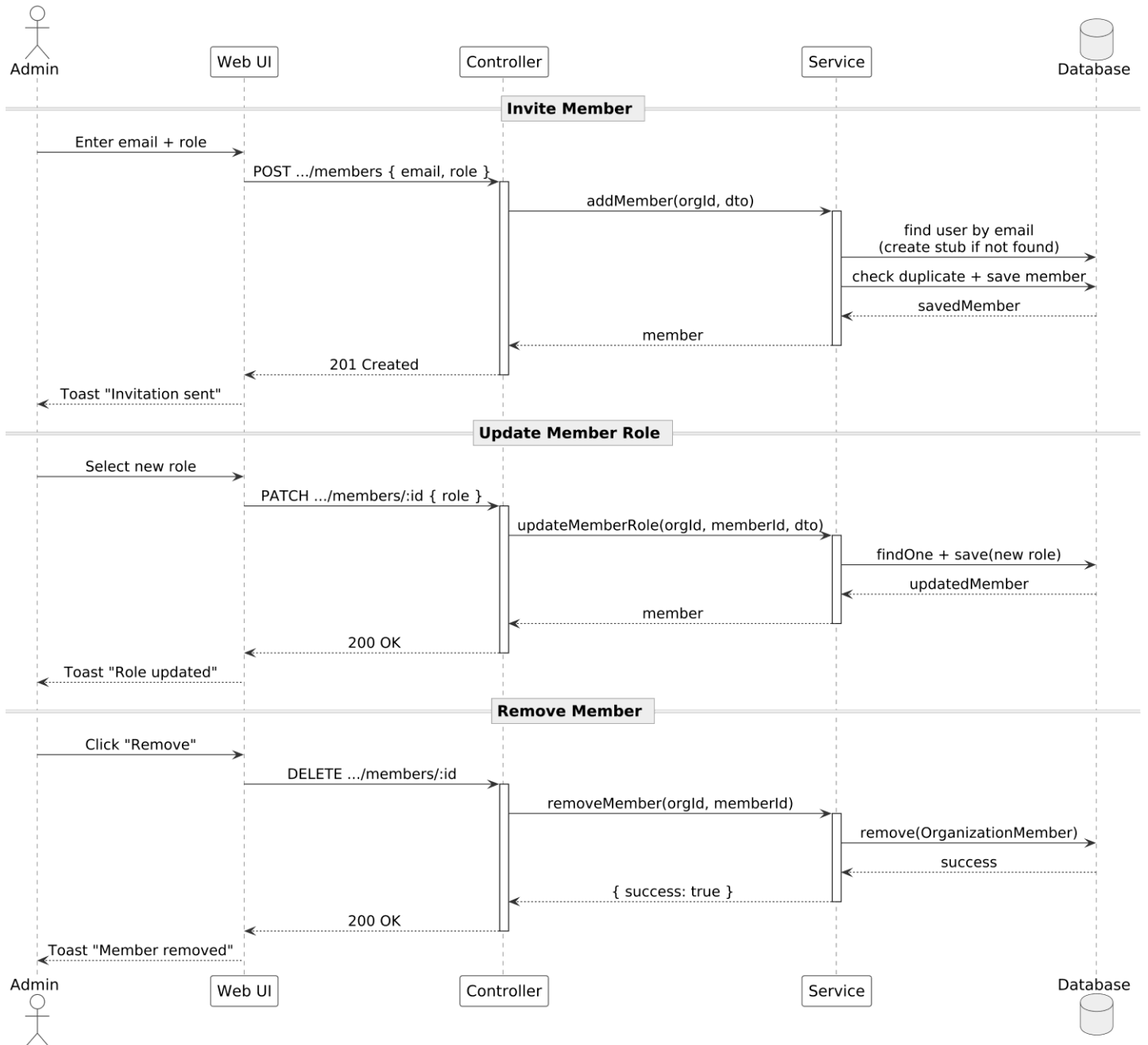


Figure 13 : Sequence Diagram: User Management

Figure 14 Shows the full asset lifecycle — creating single or bulk assets with duplicate detection, editing, and deletion guarded against active scans.

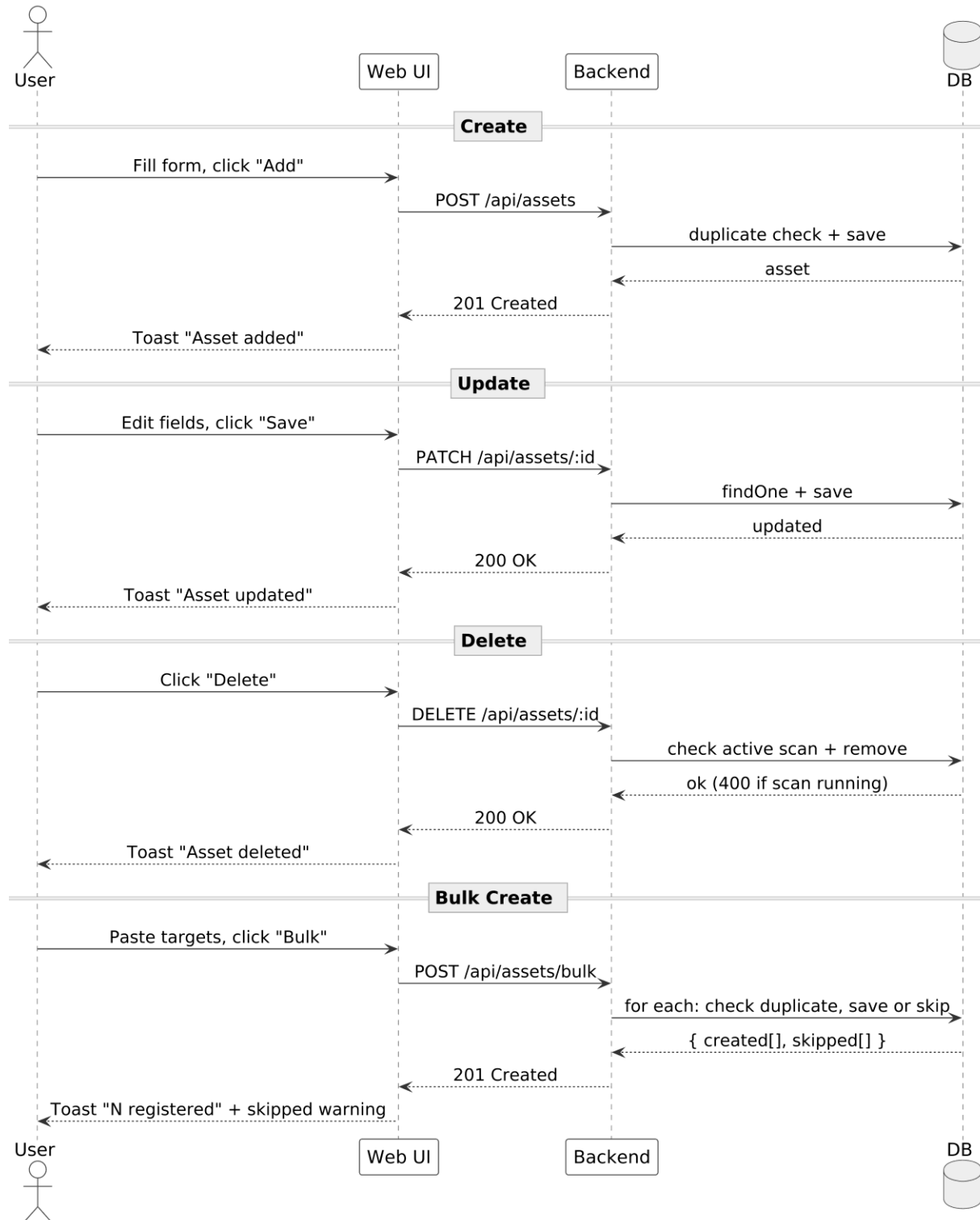


Figure 14 : Sequence Diagram: Asset Management

Figure 15 Shows the Quick Scan flow across four phases: auth discovery, infrastructure checks, vulnerability testing, and WAF detection — with live progress pushed to the browser via SSE.

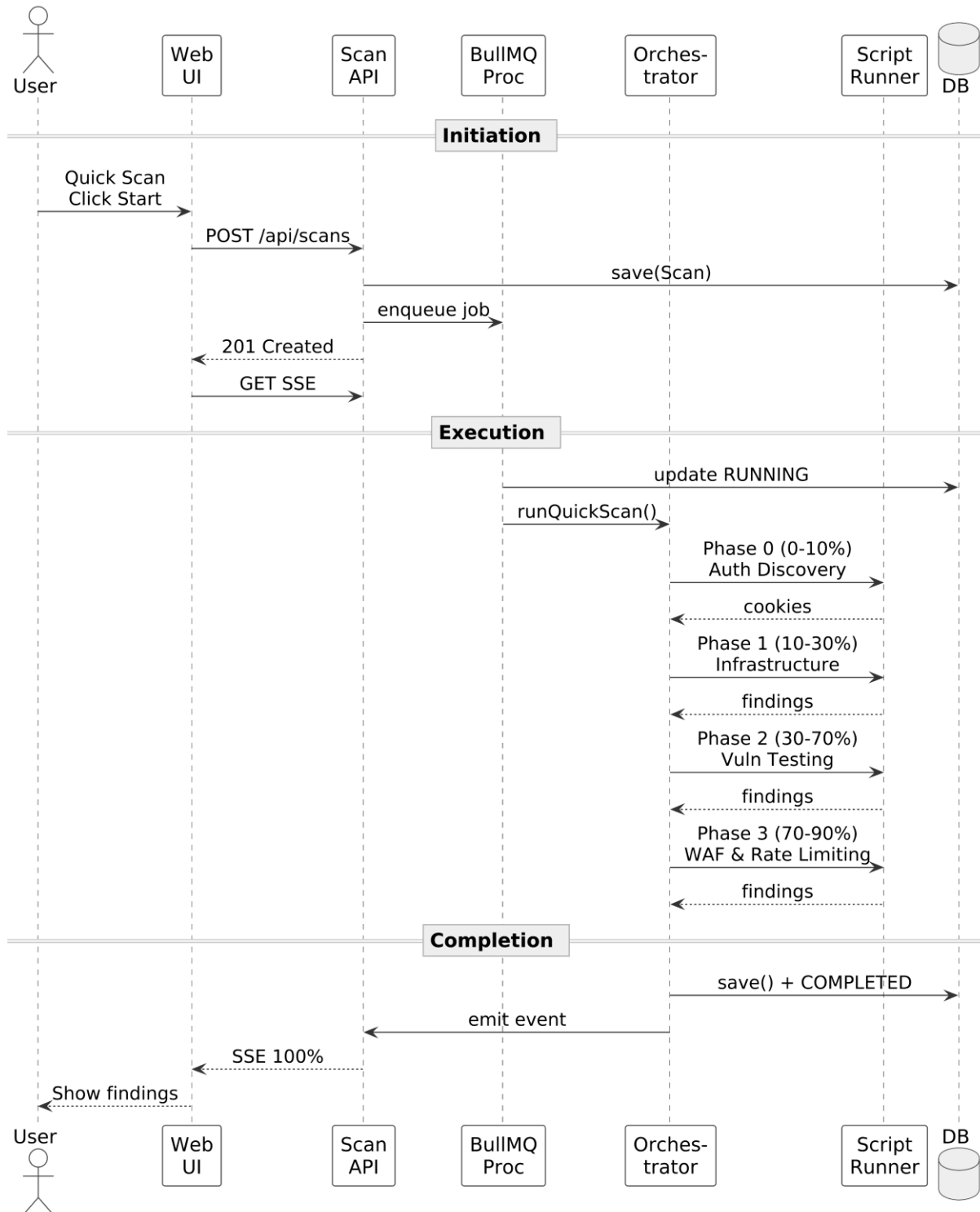


Figure 15 : Sequence Diagram: Quick Scan

Figure 16 Shows the Deep Scan flow across seven phases including port scanning, DNS recon, injection testing, access control, and protocol attacks — with live progress pushed via SSE.

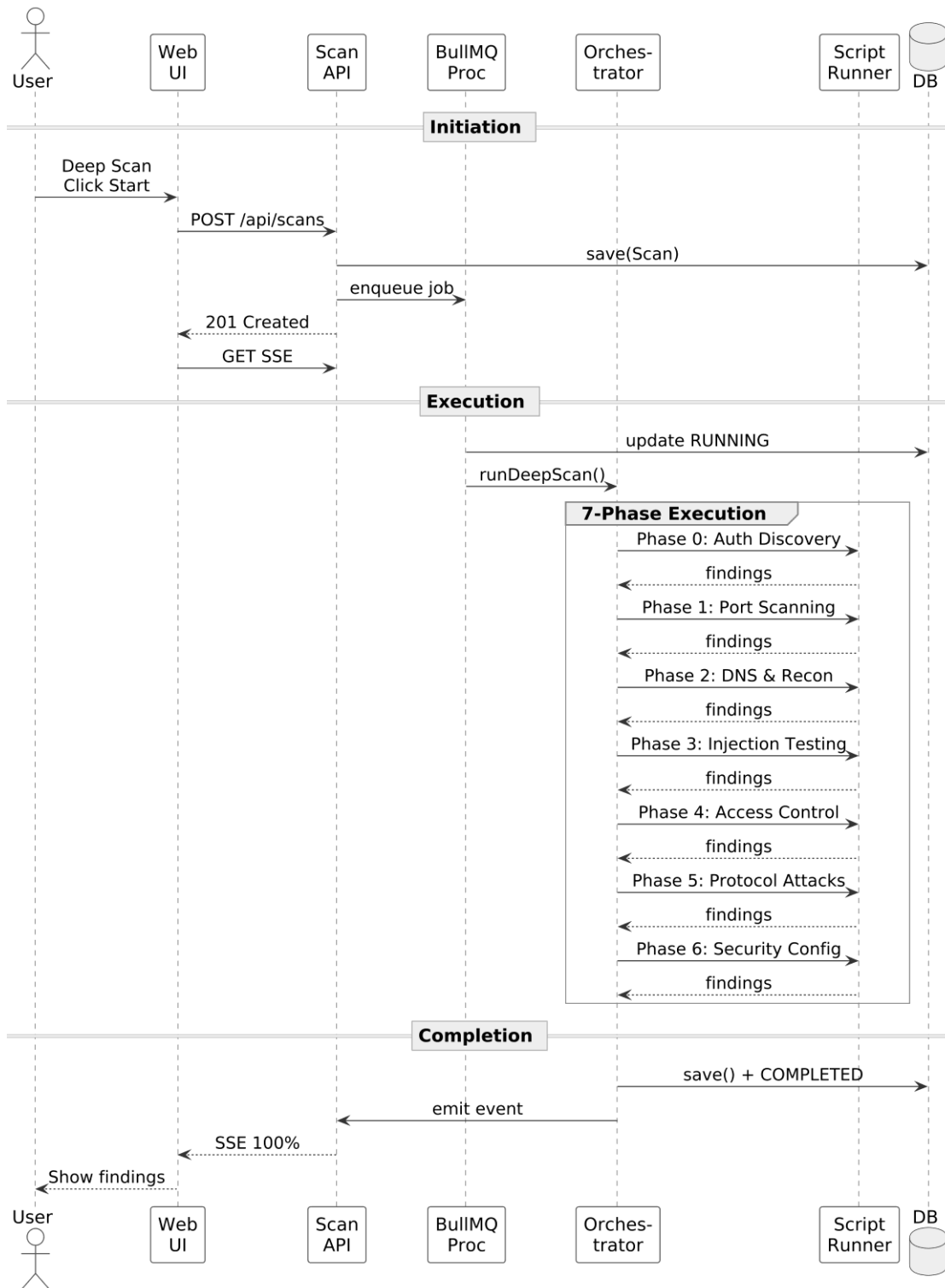


Figure 16 : Sequence Diagram: Deep Scan

Figure 17 Shows schedule creation with automatic next-run calculation, and the planned CRON-based auto-trigger that fires scans without user interaction.

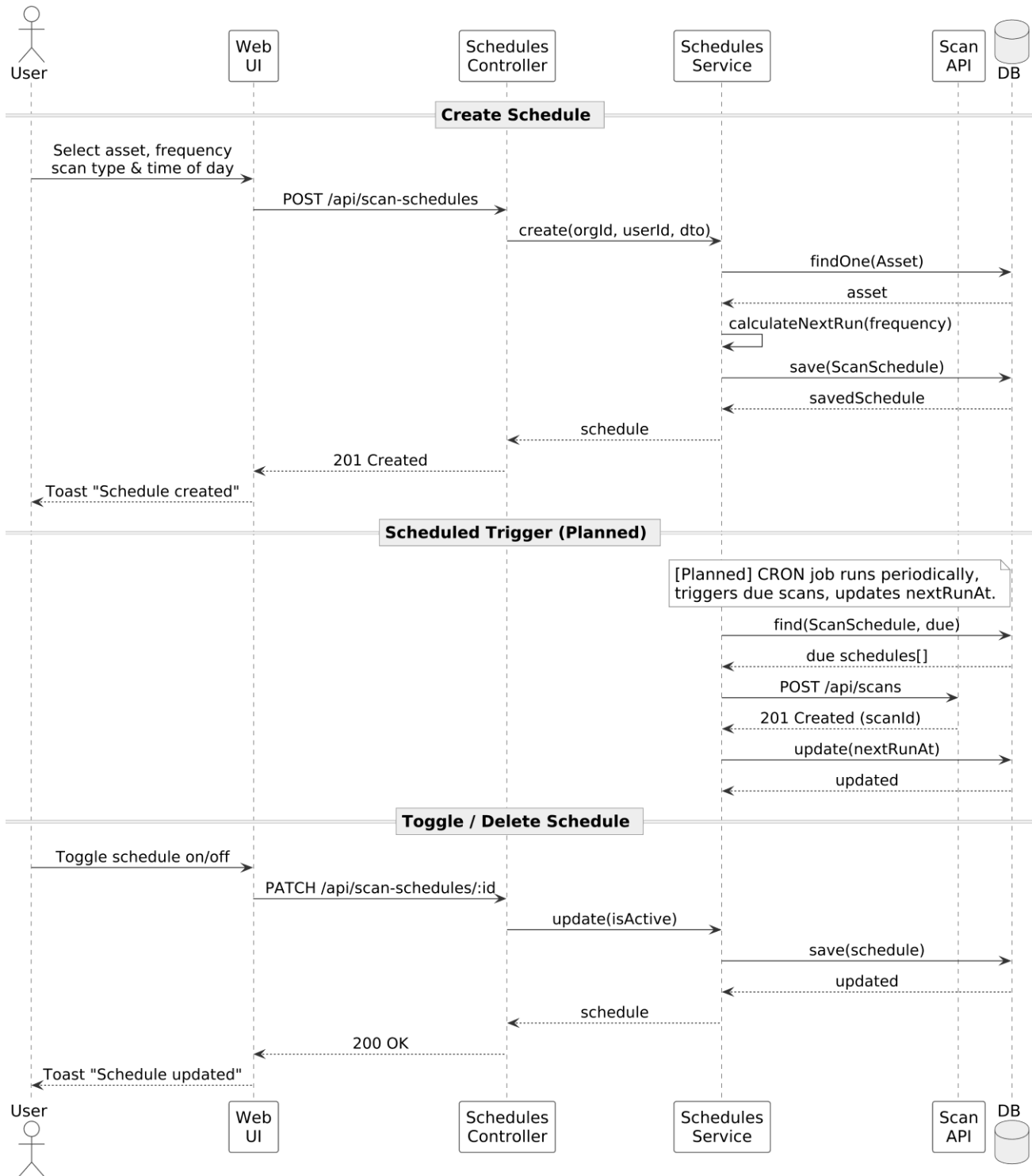


Figure 17 : Sequence Diagram: Scan Scheduling

Figure 18 Shows report generation from a completed scan — rendering findings and AI analysis into PDF, HTML, or JSON — and the download flow with a 24-hour file expiry.

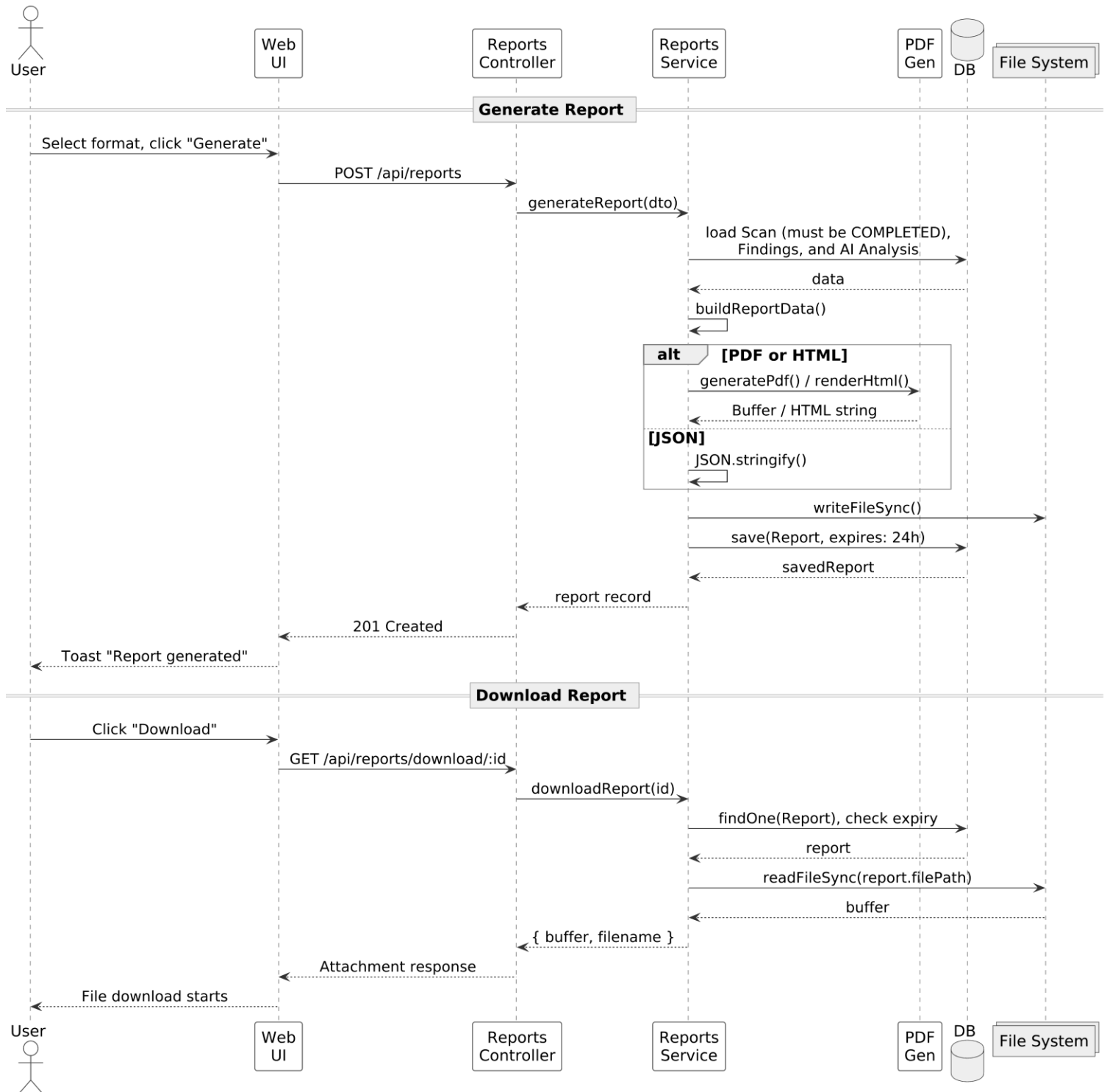


Figure 18: Sequence Diagram: Report Generation

Figure 19 Shows event-driven notification creation on scan and AI completion, along with fetching, single mark-as-read, and bulk mark-all-as-read.

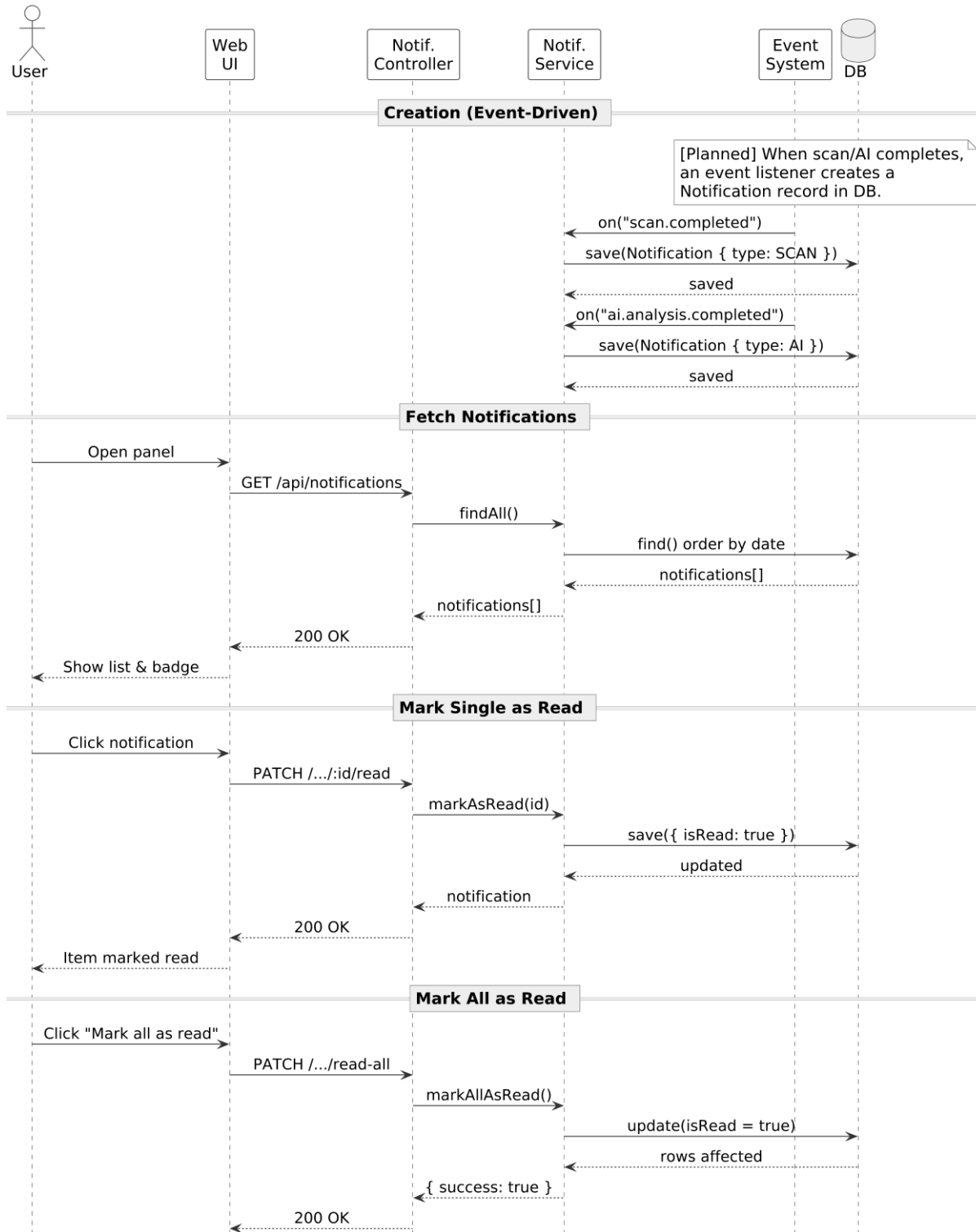


Figure 19 : Sequence Diagram: Notifications

Figure 20 Shows automatic AI analysis triggered in the background after every scan — building a Gemini prompt from findings, parsing the response into a risk score and recommendations, and storing the result.

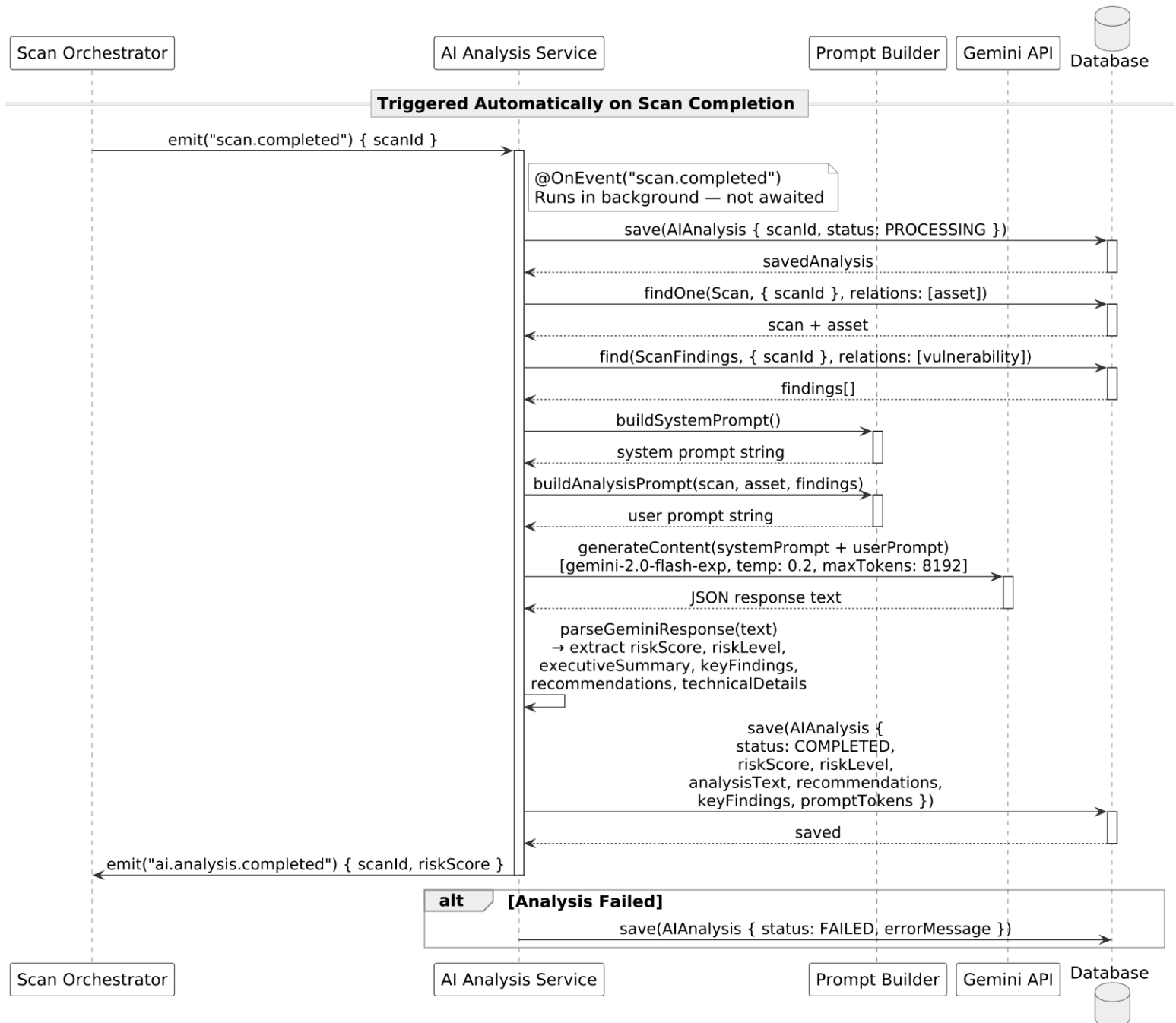


Figure 20: Sequence Diagram: AI Analysis Generation

Figure 21 Shows the TRACE AI chat flow — opening a floating widget, sending the full conversation history with org and scan context to Gemini, and displaying the reply with markdown formatting.

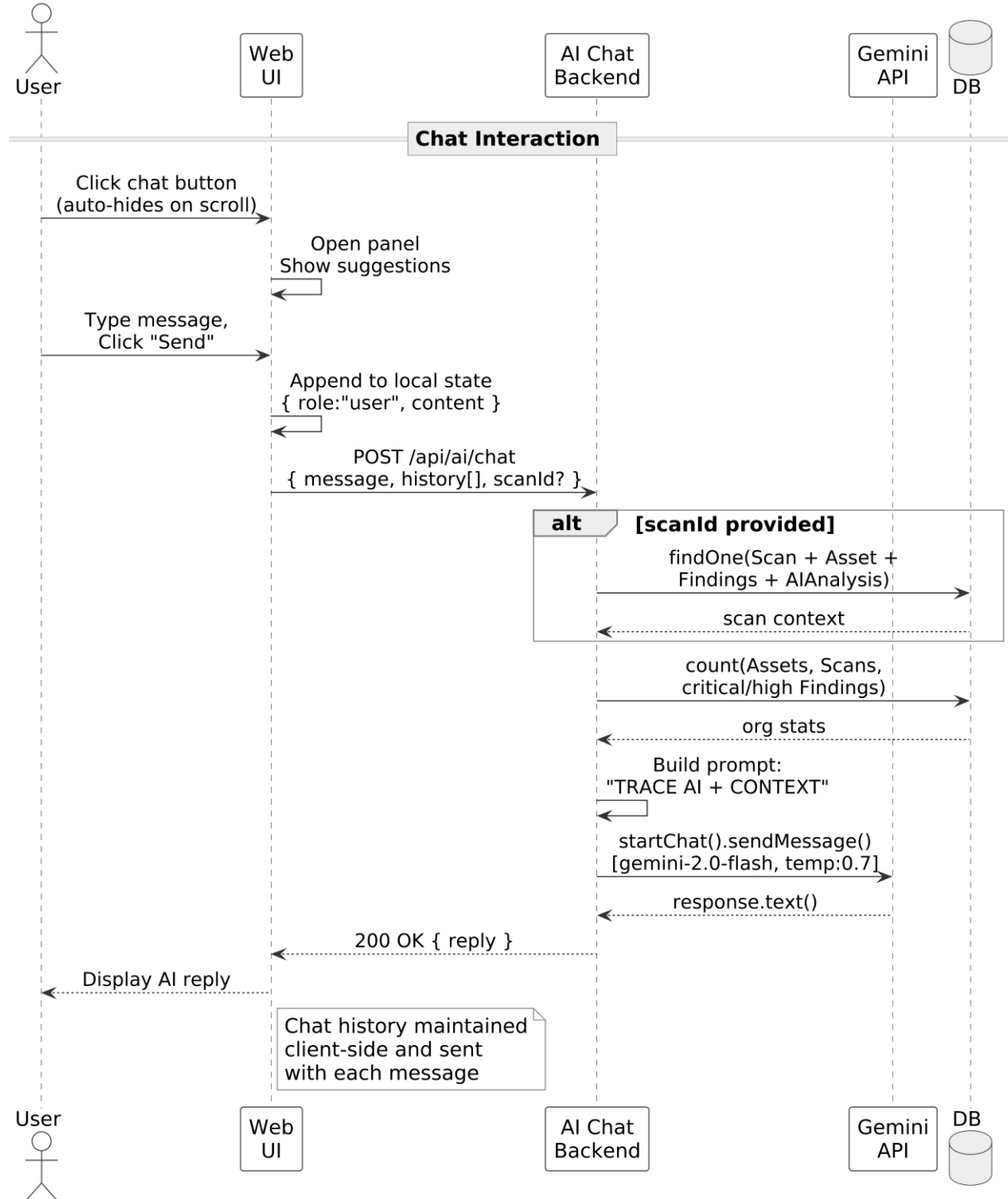


Figure 21: Sequence Diagram: AI Chat

Figure 22 Shows on-demand remediation guide generation for a specific finding — querying Gemini with vulnerability details and evidence to produce step-by-step fix instructions with code examples.

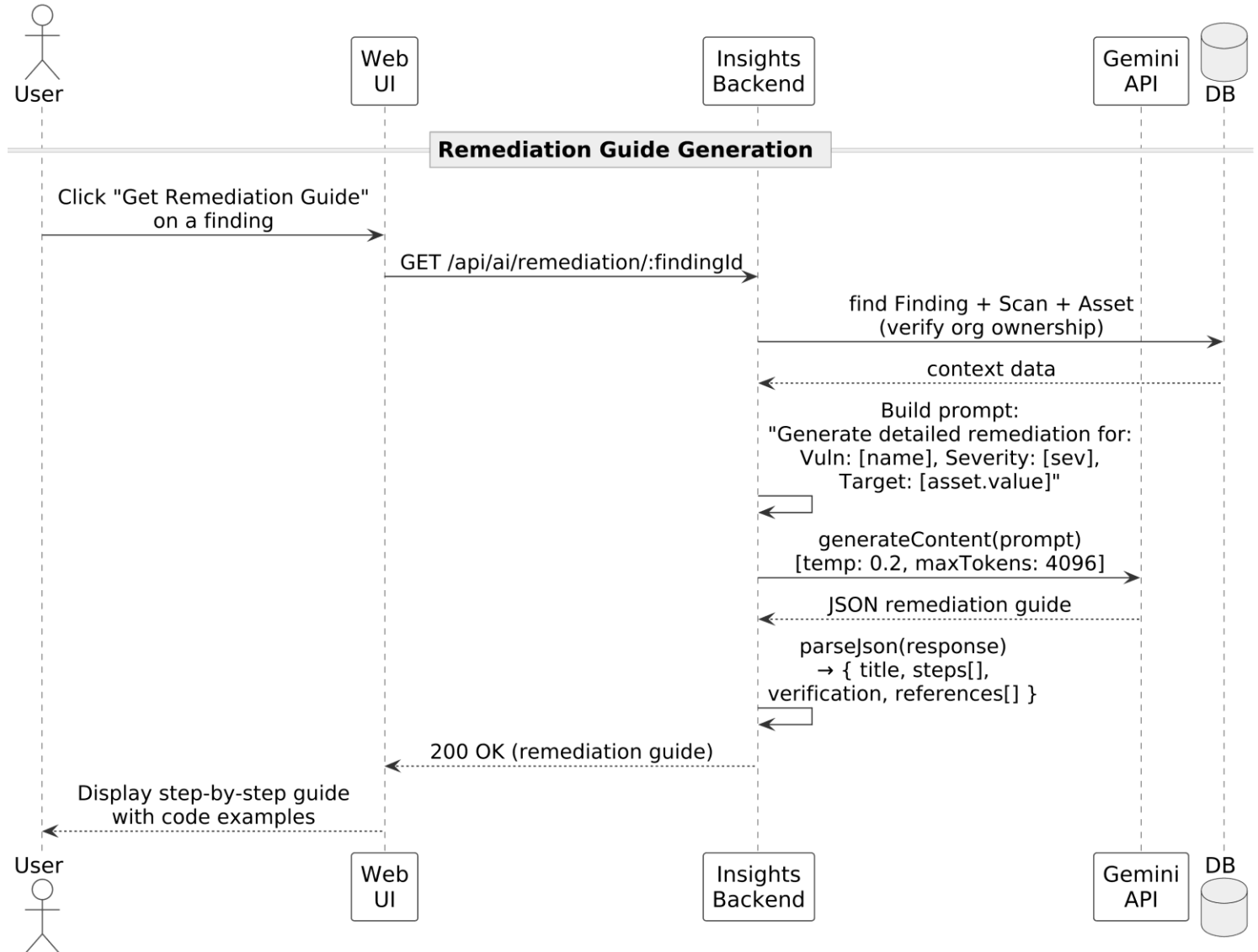


Figure 22 : Sequence Diagram: Remediation Plan Generation

3.3.4 Entity Relationship Diagram (ERD)

Figure 23 illustrates the Entity Relationship Diagram (ERD) of TRACE, showing the data model and relationships between the core entities in the PostgreSQL database. The system consists of the following primary entities:

- **User:** Stores user credentials (name, email, password hash) and timestamps. Related to OrganizationMember, Notification, and Scan entities.
- **Organization:** Represents a tenant/workspace. Contains name and settings. Related to OrganizationMember, Asset, and Scan entities.
- **OrganizationMember:** Junction entity linking Users to Organizations with a role field (ADMIN, EDITOR, VIEWER).
- **Asset:** Stores digital assets (name, type, value). Types include DOMAIN, IP, URL, and CIDR. Belongs to an Organization.
- **Scan:** Records scan configurations including type (QUICK/DEEP), status (PENDING/RUNNING/COMPLETED/FAILED/CANCELLED), and timestamps. Related to Asset, ScanFinding, AIAAnalysis, and Report.
- **ScanFinding:** Junction entity linking a Scan to a Vulnerability, storing location-specific evidence.
- **Vulnerability:** Stores unique vulnerability definitions with name, severity, category, and description.
- **AIAAnalysis:** Stores AI-generated analysis results including risk score, risk level, executive summary, key findings (JSONB), recommendations (JSONB), and token usage.
- **Notification:** User notifications with type, title, message, read status, and metadata.
- **Report:** Generated report records with format (PDF/HTML/JSON), file path, and size.
- **ScanSchedule:** Recurring scan configuration with frequency, day, time, and active status.

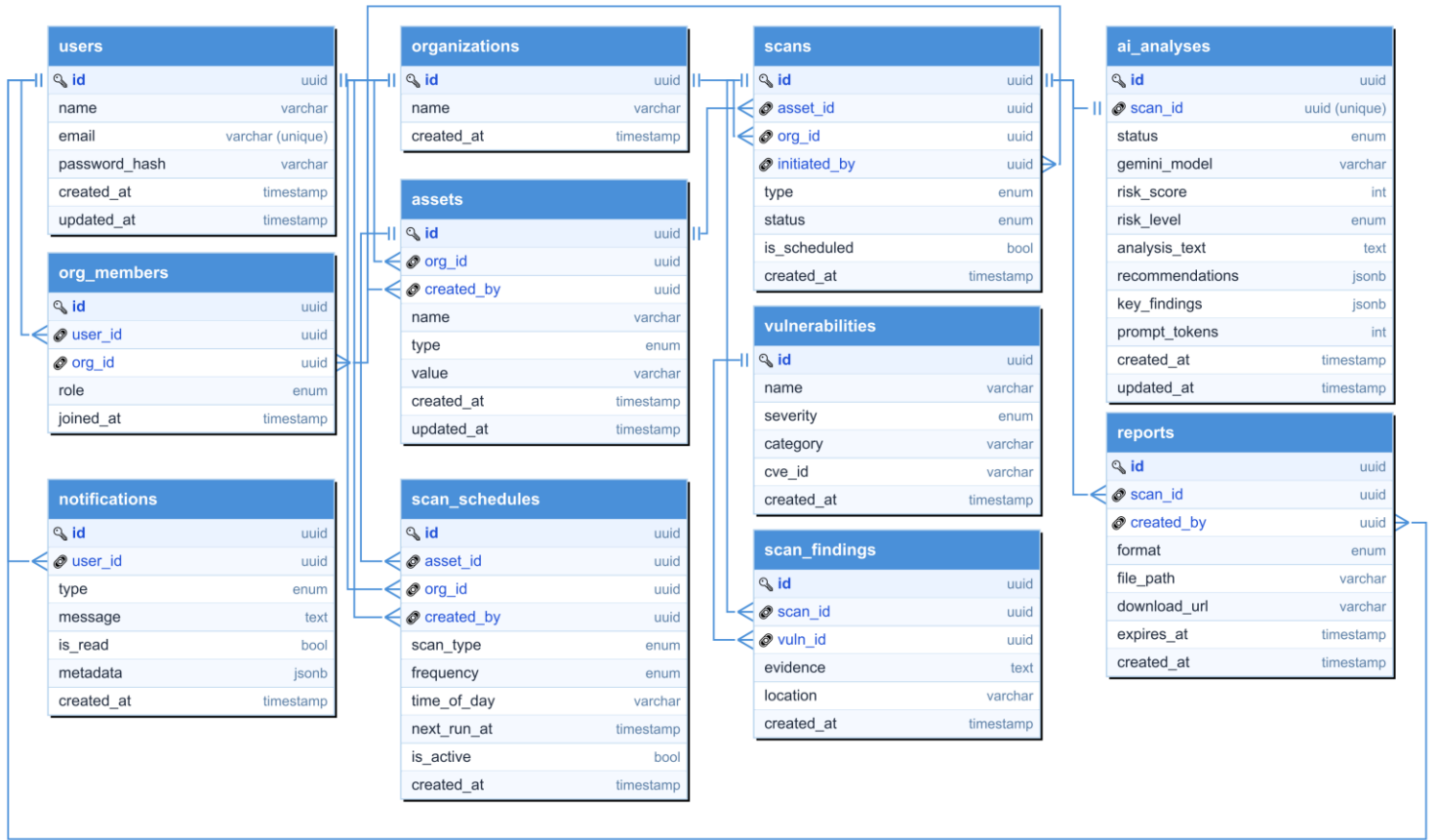


Figure 23: Entity Relationship Diagram (ERD)

3.4 System Architecture

3.4.1 Technologies Overview

The development of TRACE relies on a combination of modern technologies selected for their performance, reliability, and developer productivity. This section provides an overview of each technology used in the platform.

1. Next.js 15 (App Router)

Next.js [8] is a React-based framework for building full-stack web applications. TRACE uses Next.js 15 with the App Router architecture, which provides server-side rendering, file-based routing with route groups, and React Server Components. The App Router enables clean separation between authentication routes and dashboard routes using route groups like (auth) and (dashboard).

2. React 19

React is a declarative JavaScript library for building user interfaces through reusable components. TRACE leverages React 19's latest features for building a responsive, interactive dashboard with real-time data updates.

3. NestJS

NestJS [4] is a progressive Node.js framework for building efficient, reliable server-side applications. It uses TypeScript and follows an Angular-inspired modular architecture with decorators, dependency injection, and a well-defined module system. TRACE's backend is organized into 12 feature modules (auth, assets, scans, findings, ai-analysis, reports, etc.).

4. TypeORM

TypeORM [9] is an Object-Relational Mapping library for TypeScript that simplifies database interactions. TRACE uses TypeORM with PostgreSQL for entity management, migrations, and query building with decorator-based entity definitions.

5. PostgreSQL 15

PostgreSQL [11] is a powerful, open-source relational database system. TRACE uses PostgreSQL 15 as its primary data store for users, organizations, assets, scans, vulnerabilities, and AI analysis results. JSONB columns are used for flexible storage of AI analysis data (key findings, recommendations).

6. Redis 7

Redis [12] is an in-memory data store used by TRACE as the backing store for BullMQ job queues. It manages scan job scheduling, processing, and status tracking with high throughput and low latency.

7. BullMQ

BullMQ [6] is a premium message queue for Node.js built on Redis. TRACE uses BullMQ to manage scan jobs as background tasks, providing job scheduling, retry logic, concurrency control, and progress tracking. Each scan request creates a job in the scan-queue that is processed by the ScanProcessor.

8. Python 3.11+

Python is used for the 43+ vulnerability scanning scripts in TRACE's scan engine. Each script is an independent Python program that receives a target via command-line arguments and outputs findings as a JSON array to stdout, following the Runner Contract pattern.

9. Google Gemini AI

Google Gemini [5] is a large language model API used by TRACE for intelligent security analysis. The platform uses the gemini-2.0-flash-exp model to analyze scan results and generate structured security assessments including risk scores, executive summaries, and remediation recommendations.

10. Puppeteer

Puppeteer [10] is a Node.js library for controlling headless Chrome/Chromium. TRACE uses Puppeteer to generate professional PDF reports from HTML templates, producing print-ready security assessment documents with severity badges, charts, and structured findings.

3.4.2 System Architecture Diagram

Figure 24 illustrates the high-level system architecture of TRACE, showing the interaction between the five primary layers: Client, Frontend, Backend, Data, and External Services.

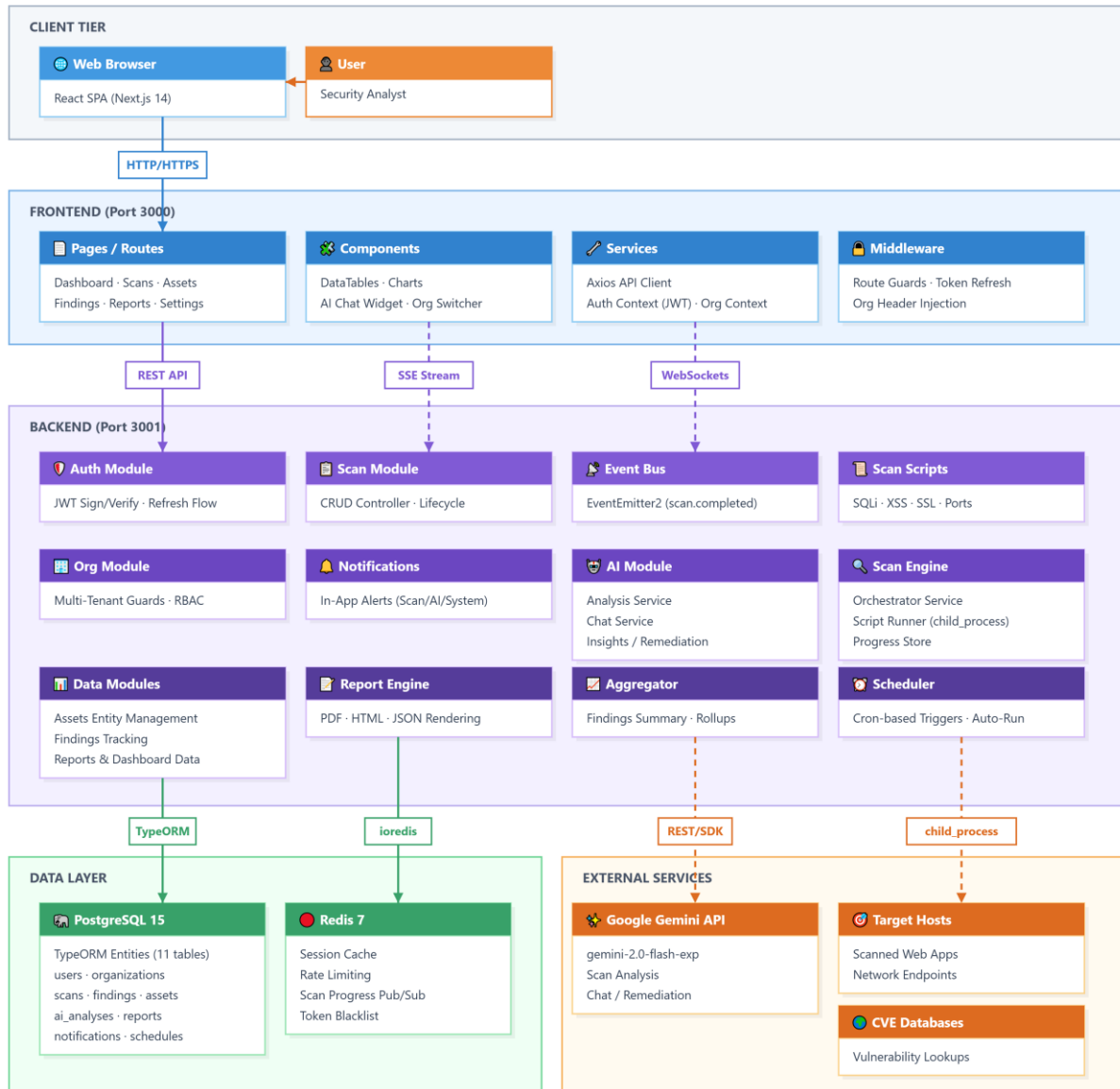


Figure 24 : System Architecture Diagram

The architecture follows a layered design:

- **Client Layer:** Security analysts and users interact with TRACE through a web browser accessing the Next.js React SPA.
- **Frontend Layer:** Next.js serves the presentation layer on Port 3000, handling the dashboard UI, specialized components, contextual services, and middleware for route guarding. It communicates via REST API, SSE Streams.
- **Backend Layer:** NestJS processes API requests on Port 3001, manages JWT authentication and RBAC, coordinates background events via an internal event bus, and drives the core scan engine, scheduler, and AI module.
- **Data Layer:** PostgreSQL 15 stores persistent relational data mapped via TypeORM. Redis 7 handles high-performance transient data, including session caching, rate limiting, and scan progress Pub/Sub.
- **External Services:** Google Gemini API for deep scan analysis, network endpoint connections during target host scanning, and outbound queries to CVE databases.

Chapter 4

Implementation and Results

CHAPTER 4: IMPLEMENTATION AND RESULTS

4.1 User Interface Design

4.1.1 Design System

TRACE employs a modern, dark-themed design system built with Tailwind CSS v4 and shadcn/ui component library. The design prioritizes clarity, accessibility, and professional aesthetics suitable for security-focused applications.

- **Primary Color Palette:** Deep navy (#1a1a2e) backgrounds with cyan/blue accent colors for interactive elements and severity-based color coding (Red for Critical, Orange for High, Yellow for Medium, Blue for Low).
- **Typography:** Inter font family for clean readability across all screen sizes.
- **Component Library:** shadcn/ui provides accessible, customizable React components including data tables, forms, dialogs, and navigation elements.
- **Responsive Layout:** Sidebar-based navigation with collapsible menu for mobile viewports.

TRACE Design System

AI VULNERABILITY SCANNER / SHARP CYBERSECURITY UI

TYPOGRAPHY

Headline 1

48 / 56px - bold

Headline 2

32 / 40px - bold

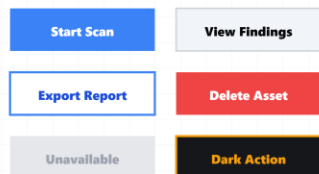
Headline 3

24 / 32px - bold

Body text for scan findings, dashboard cards, and reports

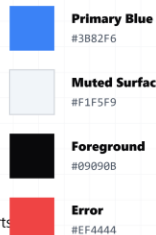
MONO_LABEL / 12px / 16px

BUTTONS / STATES

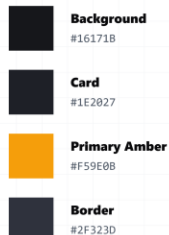


COLOR PALETTE

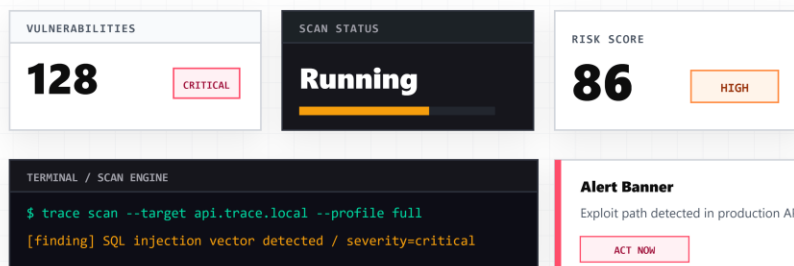
LIGHT THEME



DARK THEME



SHARP DASHBOARD UI ELEMENTS



SEVERITY BADGES



Figure 25 : TRACE Design System

4.1.2 Screen-by-Screen Breakdown

4.1.2.1 Login Page

The login page provides email/password authentication with form validation, error messaging, and a link to the registration page.

TRACE SECURITY INTELLIGENCE

SECURE YOUR DIGITAL ASSETS.

Enterprise-grade vulnerability scanning powered by artificial intelligence.

- AI-Powered Analysis**
Advanced machine learning models analyze vulnerabilities and prioritize remediation.
- 20+ Security Checks**
Comprehensive scanning including SQL injection, XSS, CSRF, and more.
- Real-time Monitoring**
Continuous monitoring with instant alerts when new threats are detected.
- Compliance Reports**
Generate detailed reports for OWASP, PCI DSS, and custom frameworks.

ADVANCED FRAMEWORKS

WELCOME BACK
Let's get you back into your security dashboard.

Email Address
admin@demo.com

Password [FORGOT?](#)
.....

☐ Remember me

[Sign In →](#)

Don't have an account? [Create one](#)

Figure 26 : Login Page

4.1.2.2 Registration Page

New users register with name, email, and password. Upon registration, a default organization is created automatically.

TRACE SECURITY INTELLIGENCE

SECURE YOUR DIGITAL ASSETS.

Enterprise-grade vulnerability scanning powered by artificial intelligence.

- AI-Powered Analysis**
Advanced machine learning models analyze vulnerabilities and prioritize remediation.
- 20+ Security Checks**
Comprehensive scanning including SQL injection, XSS, CSRF, and more.
- Real-time Monitoring**
Continuous monitoring with instant alerts when new threats are detected.
- Compliance Reports**
Generate detailed reports for OWASP, PCI DSS, and custom frameworks.

ADVANCED FRAMEWORKS

CREATE ACCOUNT
Join the elite network of security researchers.

Full Name
John Doe

Email Address
admin@demo.com

Password
Strong

Create Account →

Already have an account? [Sign in](#)

Figure 27: Registration Page

4.1.2.3 Dashboard Overview

The dashboard provides a centralized security posture view including: summary cards (total assets, active scans, open findings, critical issues), security score (0-100) with letter grade, severity distribution charts, vulnerability trends (30 days), and AI-generated security insights.

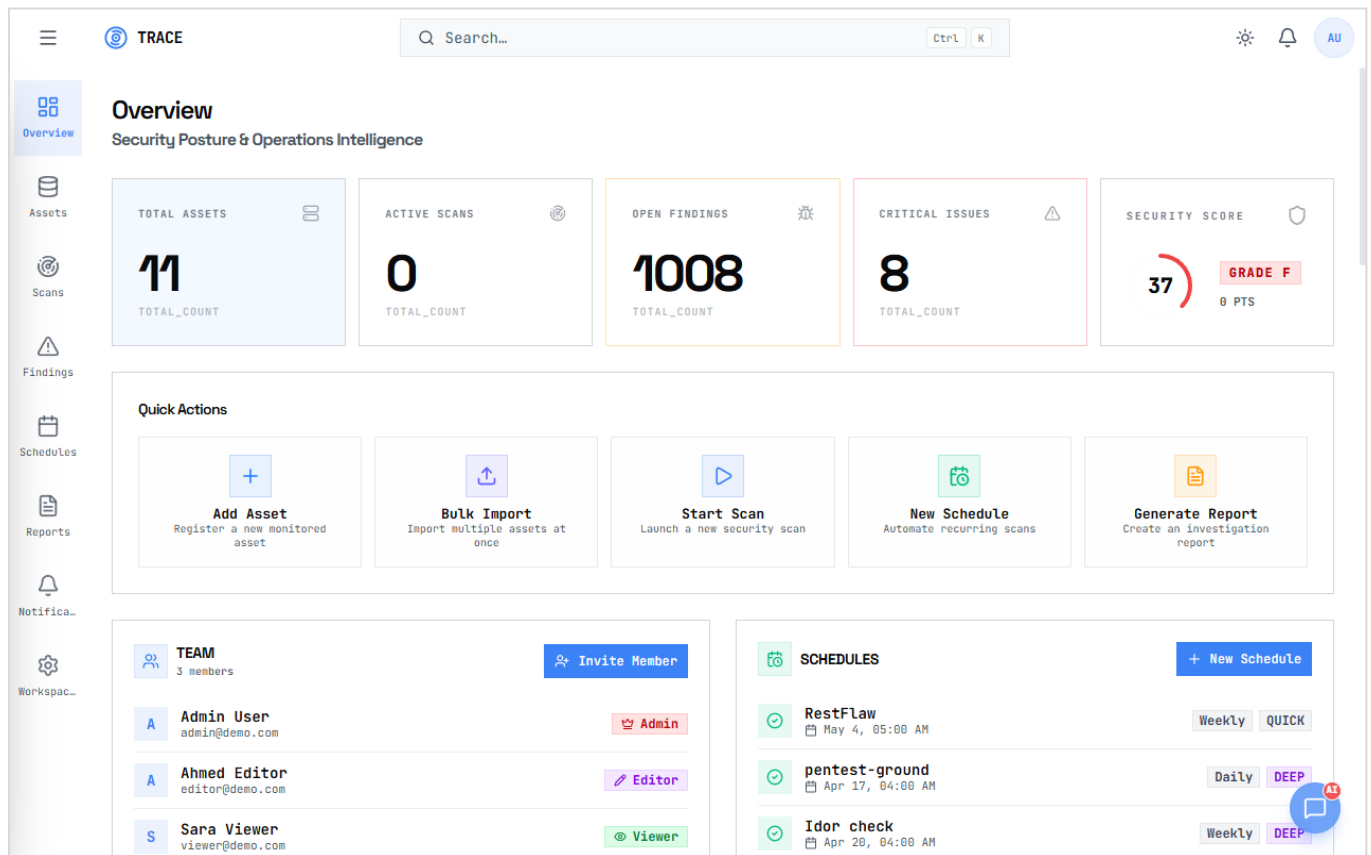


Figure 28 : Dashboard Overview

4.1.2.4 Asset Management Page

Users add, view, and delete digital assets. Supported types: Domain, IP Address, URL, and CIDR Range. Validation ensures correct format before submission.

Assets
Monitor and manage your assets

TOTAL ASSETS
11
MONITORED ASSETS

TYPE
1
DOMAINS

TYPE
2
IP ADDRESSES

RISK LEVEL
1008
CRITICAL_ISSUES

Search by name... + Type View Export Bulk + Add Asset

Name	Value	Type	Created At	Updated At	Actions
vault-scan.vercel.app	https://vault-scan.vercel.app/	URL	Apr 30	Apr 30	...
RestFlaw	https://pentest-ground.com:9000	URL	Apr 30	Apr 30	...
Linode	45.79.237.181	IP Address	Apr 29	Apr 29	...
vulnerability-scanner	https://vulnerability-scanner-mo...	URL	Apr 26	Apr 26	...
pentest-ground	https://pentest-ground.com:4280/	URL	Apr 16	Apr 16	...
Web Server - Producti...	api.example.com	Domain	Apr 16	Apr 16	...
Idor check	https://www.du.edu.eg/du_edu/pag...	URL	Apr 14	Apr 14	...

Figure 29 : Asset Management Page

4.1.2.5 Scan Initiation Page

Users select assets and scan type (Quick or Deep). Advanced settings support custom cookies and HTTP headers for authenticated scanning.

The screenshot displays the TRACE application interface for scan management. The top navigation bar includes a search bar, user profile (AU), and system icons. The left sidebar contains navigation links for Overview, Assets, Scans (active), Findings, Schedules, Reports, Notifications, and Workspaces.

Scans Dashboard:

- TOTAL SCANS:** 15 (HISTORY_COUNT)
- RUNNING NOW:** 0 (ACTIVE_PROCESSES)
- COMPLETED TODAY:** 1 (DAILY_THROUGHPUT)
- AVG. DURATION:** 2m 47s (PROCESSING_TIME)

Scan Results Table:

Asset	Type	Status	Findings Summary	Started At	Duration	Actions
vault-scan.vercel.app https://vault-scan.vercel.app/	DEEP	Completed	229	53m ago	6m 33s	...
vault-scan.vercel.app https://vault-scan.vercel.app/	DEEP	Completed	230	2d ago	6m 35s	...
RestFlaw https://pentest-ground.com:9000	QUICK	Completed	22	6d ago	1m 1s	...
vault-scan.vercel.app https://vault-scan.vercel.app/	DEEP	Completed	229	Jun 2	6m 32s	...
owasp-juice.shop https://owasp.org/www-project-juice-shop/	DEEP	Completed	54	May 30	3m 56s	...
owasp-juice.shop https://owasp.org/www-project-juice-shop/	DEEP	Completed	47	May 30	3m 5s	...
owasp-juice.shop https://owasp.org/www-project-juice-shop/	DEEP	Completed	55	May 30	4m 9s	...

At the bottom right, there is a chat icon with a red notification badge showing '15'.

Figure 30 : Scan Initiation Page

4.1.2.6 Scan Progress — Real-Time Progress Bar

A progress bar shows completion percentage with phase status messages streamed via SSE.

Users can cancel running scans.

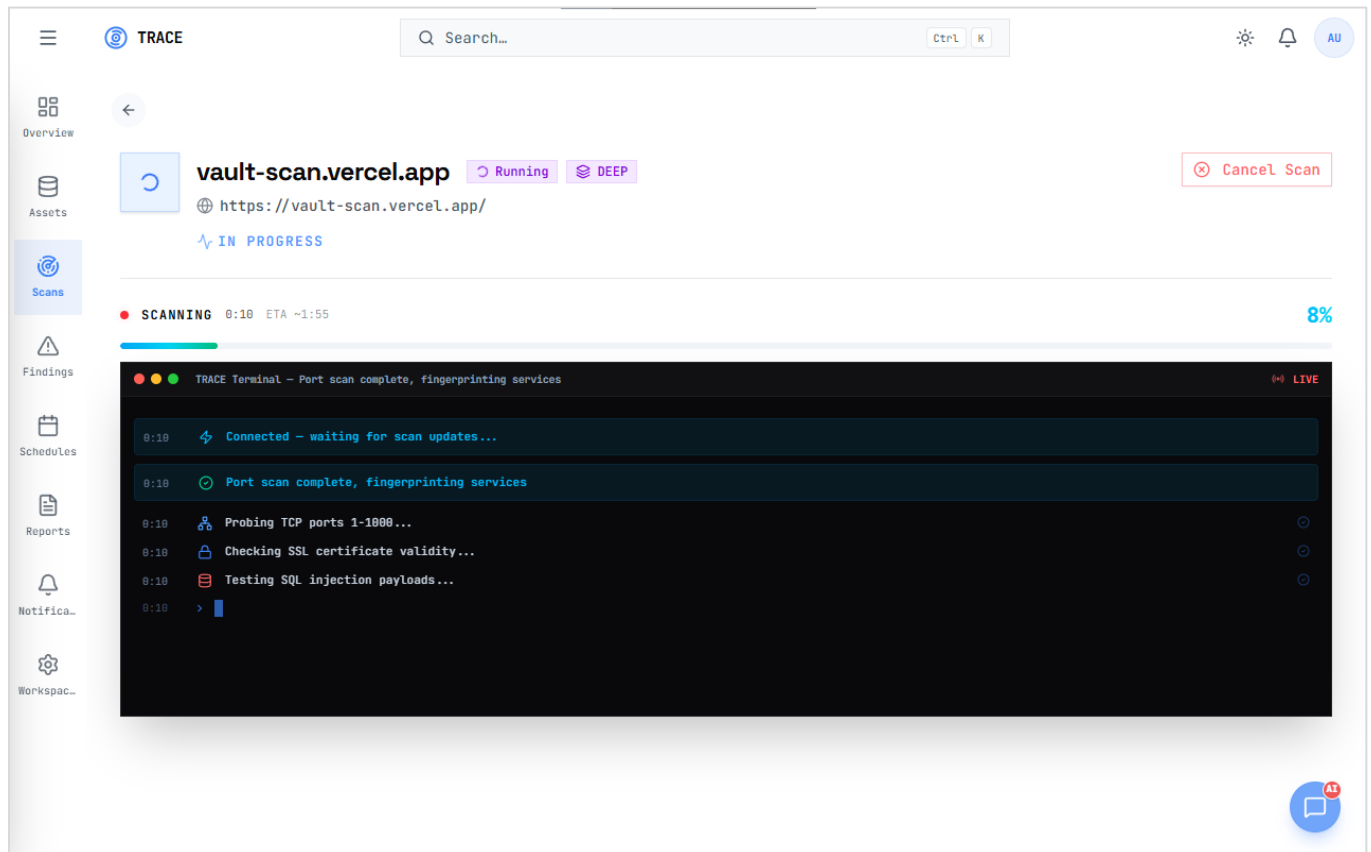


Figure 31 : Real-Time Progress Bar

4.1.2.7 Scan Results — Findings Table

Results display all detected vulnerabilities in a sortable, filterable table with severity badges, category, location, evidence, and remediation steps.

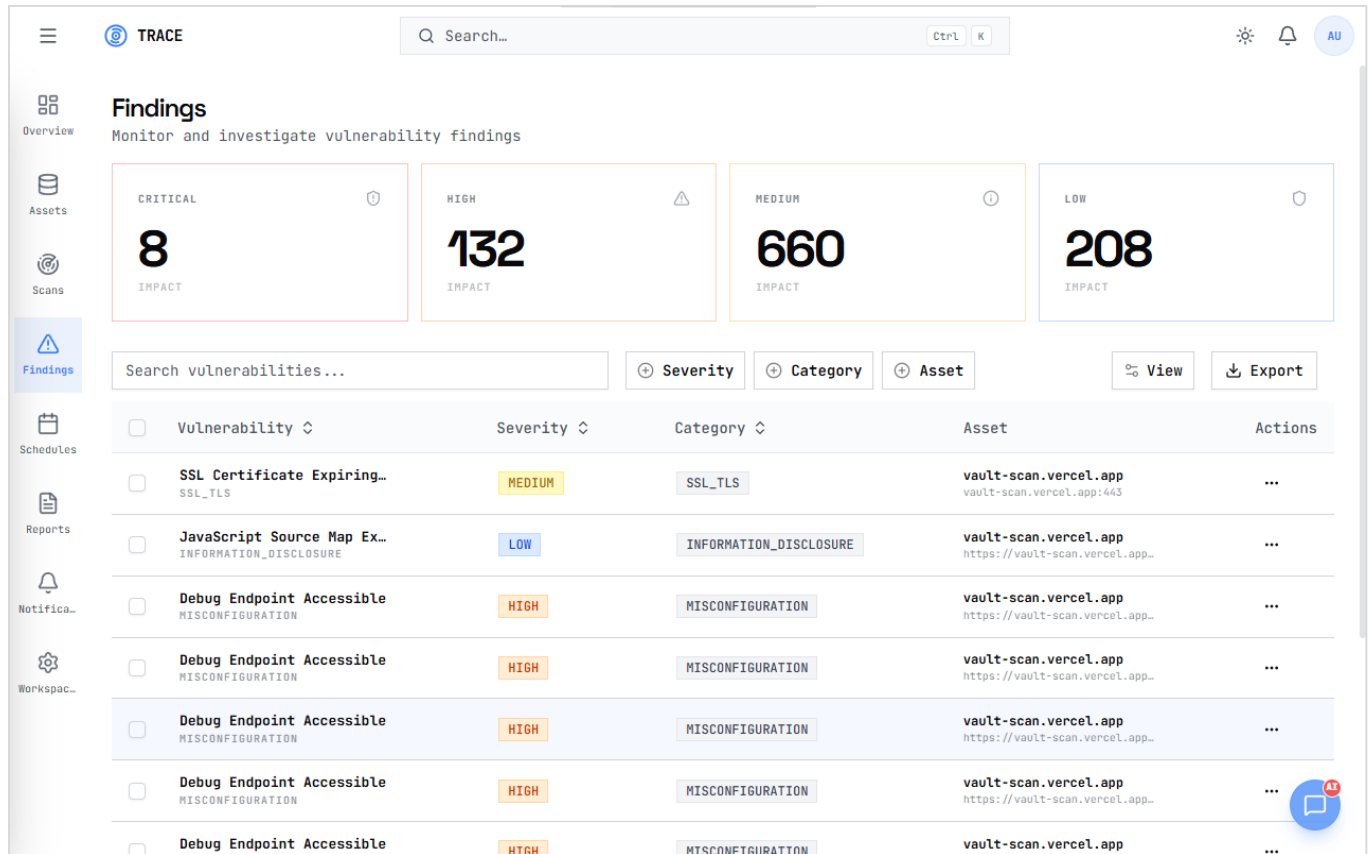


Figure 32 : Scan Results — Findings Table

4.1.2.8 AI Analysis Page

Displays the Gemini-powered analysis including risk score gauge, executive summary, key findings with impact assessment, and prioritized recommendations.

The screenshot displays the TRACE AI Analysis Page. The interface includes a sidebar with navigation options: Overview, Assets, Scans, Findings, Schedules, Reports, Notifica..., and Workspac... The main content area has tabs for Findings, AI Analysis, Raw Output, and Details. The Findings tab is active, showing a table of vulnerabilities.

VULNERABILITY	CATEGORY	LOCATION	EVIDENCE
Login Form Detected LOW	AUTH_DISCOVERY	https://vault-scan.vercel.app/Li	Login form found at
SECURITY ANALYSIS Login form found at https://pentest-ground.com:4288/login.php with fields: username, password VULNERABILITY EVIDENCE Login form found at https://vault-scan.vercel.app/login with fields: email, password REMEDIATION GUIDE Review and remediate this low severity finding. AI Fix Guide			
Open Registration Available MEDIUM	AUTH_DISCOVERY	https://vault-scan.vercel.app/r4	Registration page
Service Banner Detected LOW	SERVICE_VERSION	vault-scan.vercel.app:88	Vercel banner dete
Missing SPF Record HIGH	DNS	vault-scan.vercel.app	No SPF (v=spf1) T>

Figure 33: AI Analysis Page

4.1.2.9 AI Chat Interface

Conversational assistant for security questions, context-aware with actual scan data.

The screenshot displays the TRACE AI interface. On the left is a sidebar with navigation icons for Overview, Assets, Scans, Findings, Schedules, Reports, Notifications, and Workspaces. The main panel shows the 'Findings' tab with a table of vulnerabilities. The table has columns for 'VULNERABILITY' and 'CATEGORY'. The first finding is 'Login Form Detected' with a 'LOW' severity and 'AUTH_DISCOVERY' category. Below this, a 'SECURITY ANALYSIS' section shows the login form found at <https://pentest-ground.com:4280/login.php> with fields: username and password. A 'VULNERABILITY EVIDENCE' section shows the login form found at <https://vault-scan.vercel.app/login> with fields: email, password. A 'REMEDATION GUIDE' section provides a green box with the text: 'Review and remediate this low severity finding.' and a blue button labeled 'AI Fix Guide'. The second finding is 'Open Registration Available' with a 'MEDIUM' severity and 'AUTH_DISCOVERY' category. The third finding is 'Service Banner Detected' with a 'LOW' severity and 'SERVICE_VERSION' category. The fourth finding is 'Missing SPF Record' with a 'HIGH' severity and 'DNS' category.

On the right, the 'TRACE AI' chat window is open, titled 'Security Assistant - Scan Context'. It contains a blue button with the text 'What are the most critical issues?'. The chat message reads: 'You've got a lot of findings, but let's focus on the most critical ones first. These are the issues that pose the biggest immediate risk to your application. Here are the critical and high-severity findings from your scan: Critical Findings [CRITICAL] Default Credentials Accepted at <https://vault-scan.vercel.app/login> * Risk: This is the most severe finding. It means an attacker could log in using common default credentials (like admin:admin) without needing to guess or crack passwords. This could grant them full access to your application. * Remediation: * Immediately change any default credentials used in your application. * Implement a secure password policy for all user accounts.' At the bottom of the chat window is a text input field with the placeholder 'Ask about security...' and a send button.

Figure 34: AI Chat Interface

4.1.2.10 Report Generation Page

Generate downloadable reports in PDF, HTML, or JSON from completed scans.

The screenshot displays the TRACE Scan Automation interface. At the top, there's a search bar and a user profile 'AU'. The left sidebar contains navigation icons for Overview, Assets, Scans, Findings, Schedules, Reports, Notifications, and Workspaces. The main content area is titled 'Scan Automation' with the subtitle 'Monitor and manage your recurring scans'. It features four summary cards: 'TOTAL SCHEDULES' (5), 'STATUS' (5 ACTIVE), 'STATUS' (0 PAUSED), and 'UPCOMING SCAN' (Apr 14). Below these is a table of scheduled scans with columns: Asset, Scan Type, Frequency, Status, Next Run At, and Actions. The table lists five scans: RestFlow (QUICK, Weekly, Active), pentest-ground (DEEP, Daily, Active), Idor check (DEEP, Weekly, Active), API Server (QUICK, Daily, Active), and owasp-juice.shop (DEEP, Daily, Active). At the bottom, there are controls for 'Rows per page' (set to 10) and 'Page 1 of 1'.

Asset	Scan Type	Frequency	Status	Next Run At	Actions
RestFlow	QUICK	Weekly	Active	May 4, 2026, 05:00 AM	...
pentest-ground	DEEP	Daily	Active	Apr 17, 2026, 04:00 AM	...
Idor check	DEEP	Weekly	Active	Apr 20, 2026, 04:00 AM	...
API Server	QUICK	Daily	Active	Apr 14, 2026, 04:00 AM	...
owasp-juice.shop	DEEP	Daily	Active	Apr 14, 2026, 04:00 AM	...

Figure 35: Report Generation Page

4.1.2.11 Scan Scheduling Page

Configure recurring scans with frequency (Daily, Weekly, Monthly), day of week, and time of day.

The screenshot displays the 'Scan Automation' interface in the TRACE application. The top navigation bar includes a search bar, user profile (AU), and system icons. The left sidebar contains navigation links for Overview, Assets, Scans, Findings, Schedules (highlighted), Reports, Notifications, and Workspaces.

The main content area is titled 'Scan Automation' with the subtitle 'Monitor and manage your recurring scans'. It features four summary cards: 'TOTAL SCHEDULES' (5), 'STATUS' (5 ACTIVE), 'STATUS' (0 PAUSED), and 'UPCOMING SCAN' (Apr 14).

Below the summary cards is a table of scheduled scans. The table has columns for Asset, Scan Type, Frequency, Status, Next Run At, and Actions. The data rows are as follows:

Asset	Scan Type	Frequency	Status	Next Run At	Actions
RestFlow	QUICK	Weekly	Active	May 4, 2026, 05:00 AM	...
pentest-ground	DEEP	Daily	Active	Apr 17, 2026, 04:00 AM	...
Idor check	DEEP	Weekly	Active	Apr 20, 2026, 04:00 AM	...
API Server	QUICK	Daily	Active	Apr 14, 2026, 04:00 AM	...
owasp-juice.shop	DEEP	Daily	Active	Apr 14, 2026, 04:00 AM	...

At the bottom of the table, there is a 'Rows per page' dropdown set to 10 and a 'Page 1 of 1' indicator. A blue chat bubble icon is visible in the bottom right corner.

Figure 36 : Scan Scheduling Page

4.1.2.12 Notifications Panel

Alerts for scan completion, failures, AI analysis readiness, and critical vulnerability discoveries.

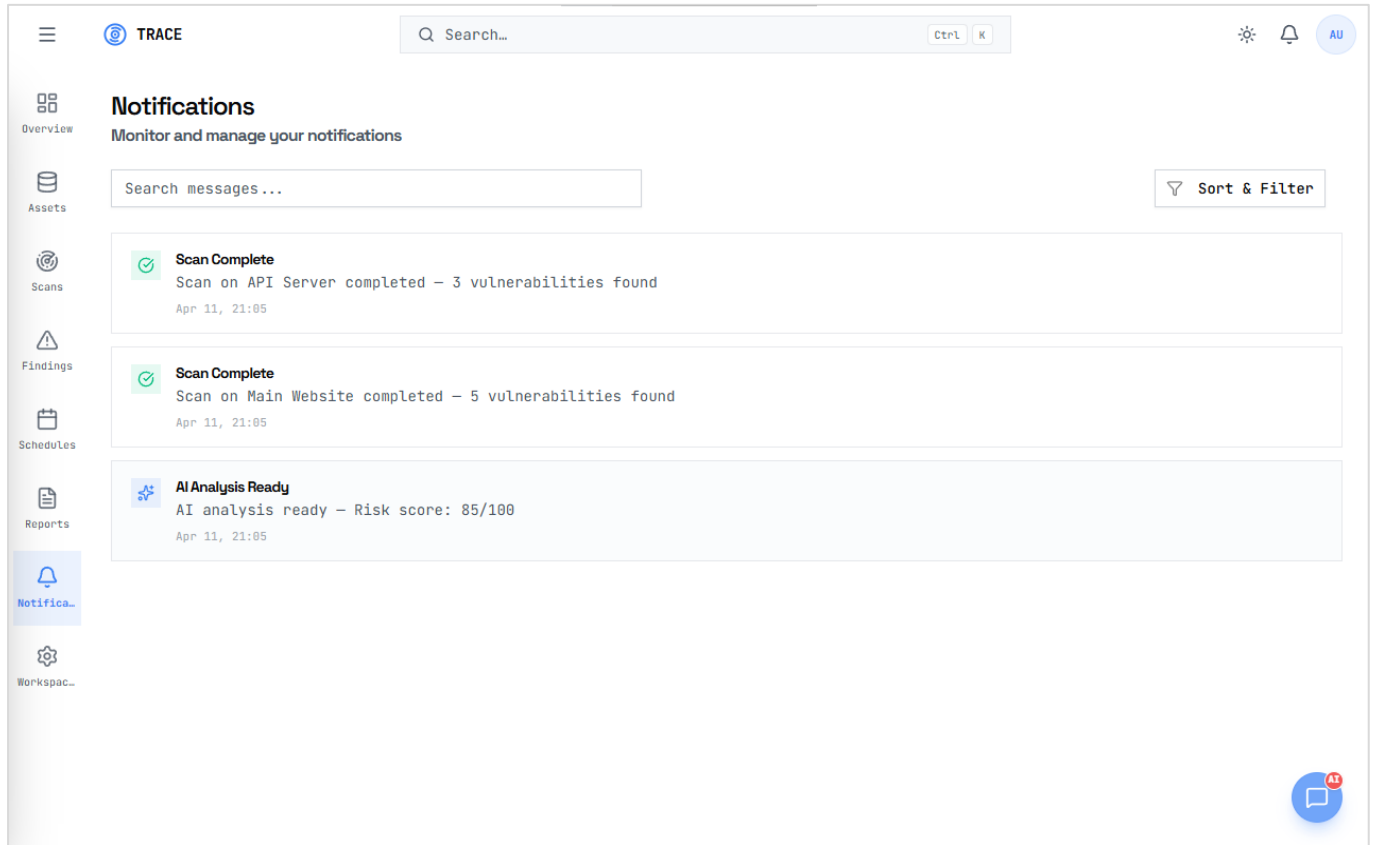


Figure 37: Notifications Panel

4.1.2.13 Organization Settings — Member Management

Administrators manage members and assign roles: Admin (full access), Editor (manage assets/scans), Viewer (read-only).

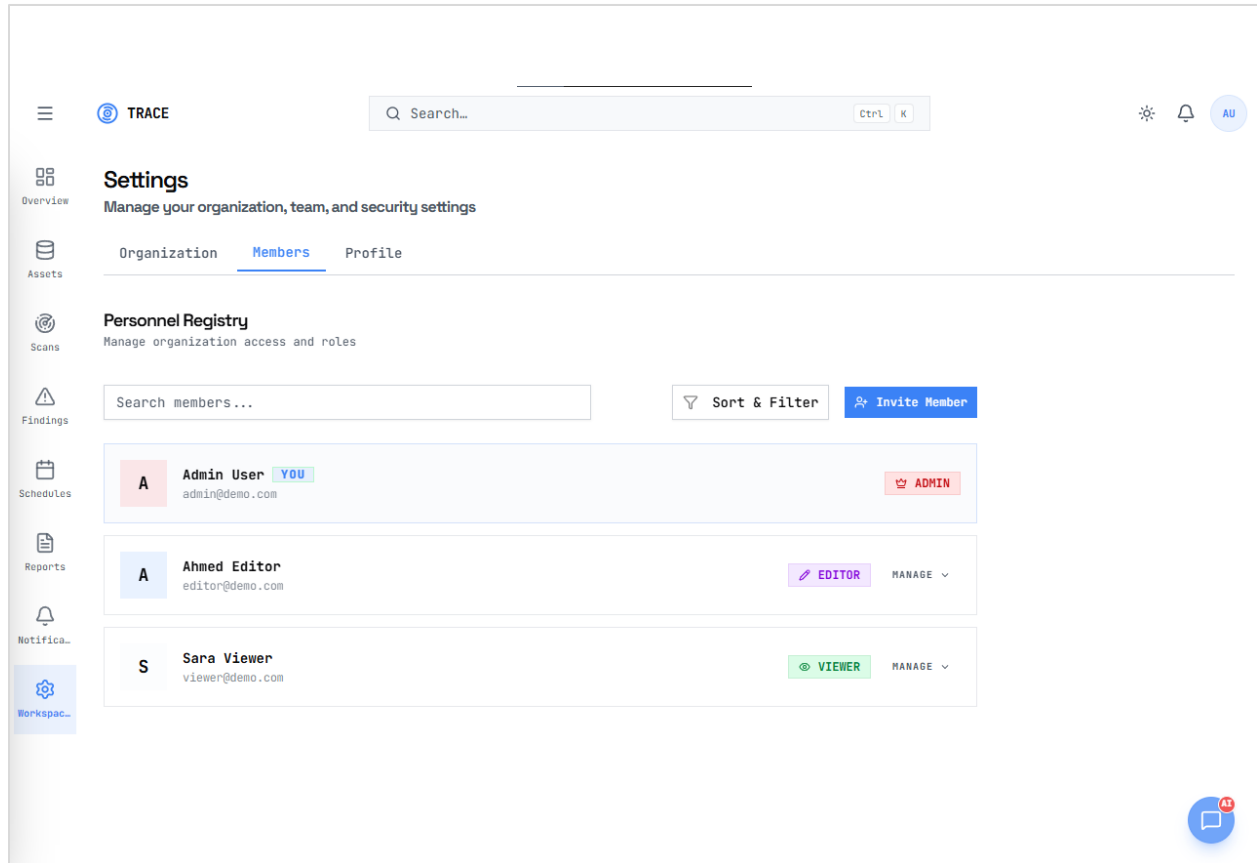


Figure 38 : Organization Settings — Member Management

4.1.2.14 Ai Remediation Plan

AI remediation plans translate scan findings into clear, prioritized action steps. Each plan explains the issue, impact, recommended fix, and validation steps so teams can resolve vulnerabilities faster with guided, AI-assisted remediation.

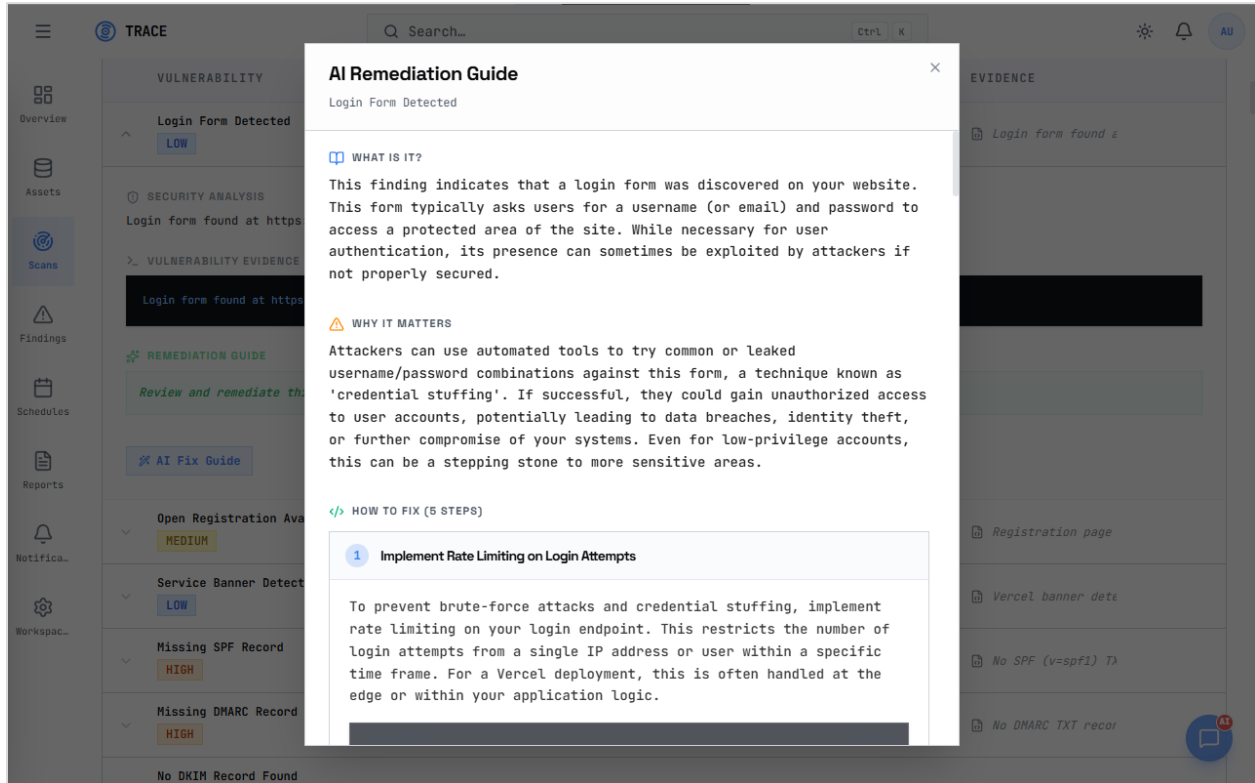


Figure 39: Ai Remediation Plan

4.1.2.15 Ai Insights

AI insights summarize scan activity, highlight risk trends, and surface recommended actions directly from the overview dashboard.

The screenshot displays the TRACE dashboard interface. On the left is a vertical sidebar with navigation icons for Overview, Assets, Scans, Findings, Schedules, Reports, and Notifications. The main content area is divided into two panels. The left panel, titled 'AI Security Insights' and 'POWERED BY GEMINI AI', contains a summary paragraph about high and critical vulnerabilities, a 'TOP PRIORITIES' section with three items: 'Address Missing Clickjacking Protection', 'Remediate Debug Endpoint Accessibility', and 'Enforce Content-Security-Policy', each with a brief description and a recommended action. The right panel, titled 'ACTIVITY FEED', shows a list of events: 'Deep scan completed' and several 'Debug Endpoint Accessible found' entries with associated URLs and timestamps (2H AGO). A search bar and user profile are at the top right.

Figure 40: Ai Insights

4.1.2.16 Responsive Design

Responsive design ensures TRACE works smoothly across desktop, tablet, and mobile screens. Layouts adapt to each device size, keeping dashboards, tables, forms, and security workflows clear, accessible, and easy to use.

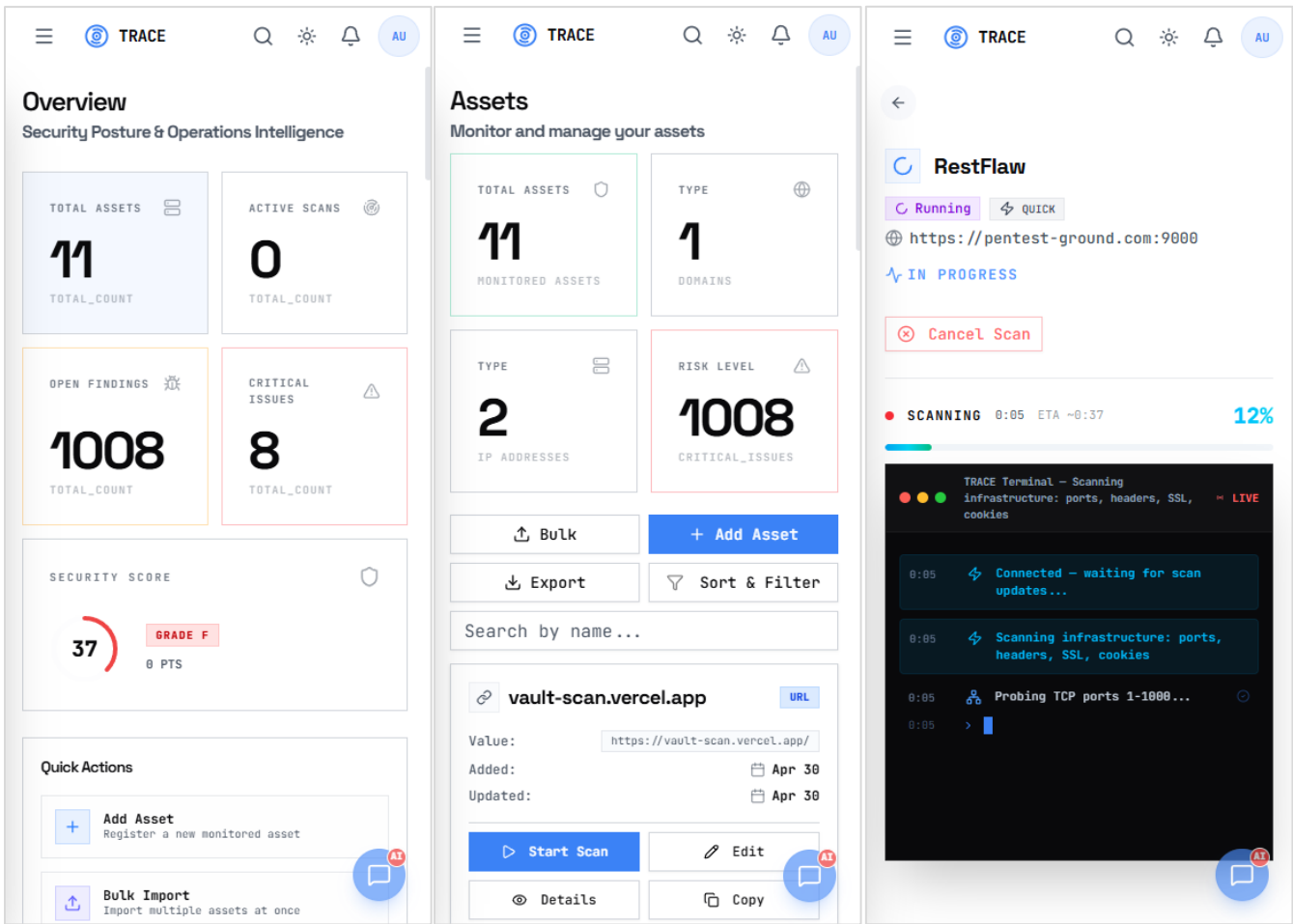


Figure 43 : Dashboard Mobile Screen

Figure 41: Assets Mobile Screen

Figure 42: Scan Progress Mobile Screen

4.1.2.17 Dark Mode

Dark mode provides a focused, low-glare interface for security teams working for long periods. TRACE adapts its colors, contrast, cards, charts, and controls to keep the dashboard readable and visually consistent.



Figure 44: Dark Landing Page

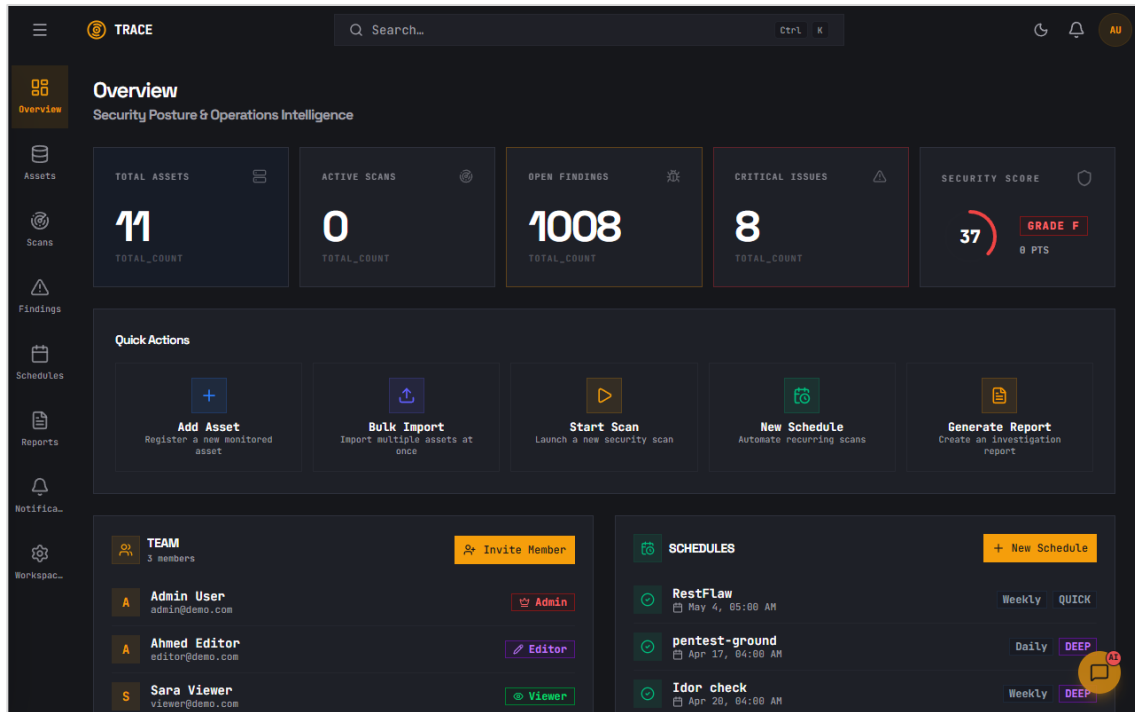


Figure 45 : Dark Dashboard

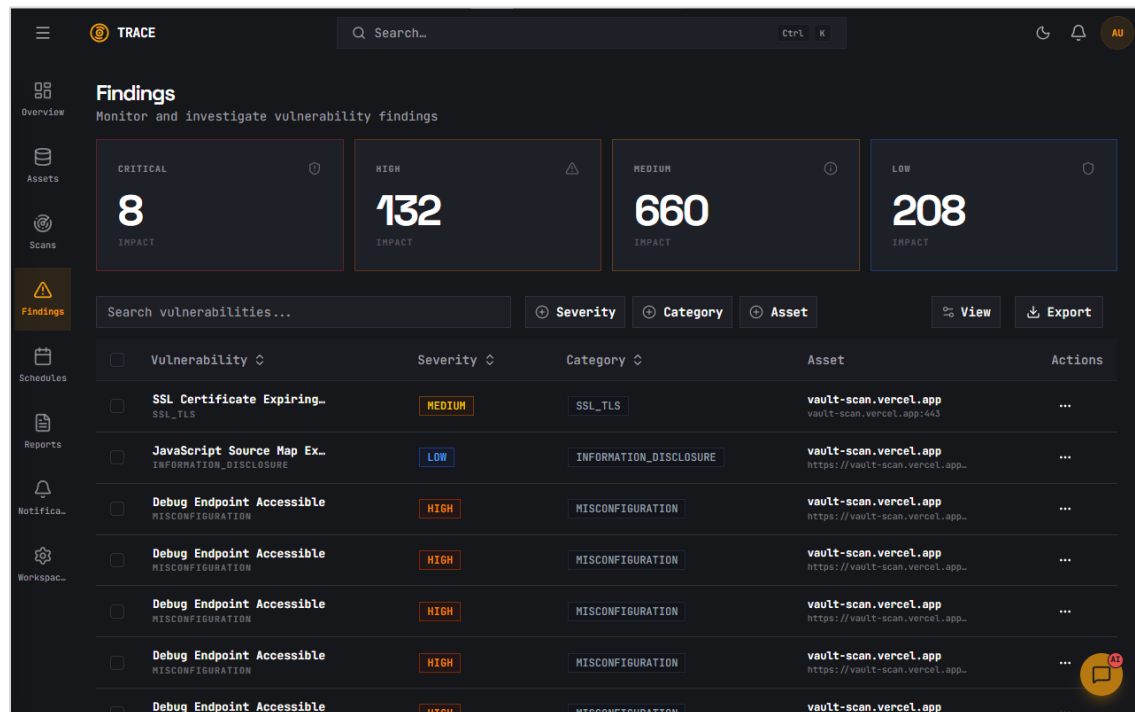


Figure 46 : Dark Findings Page

4.1.2.18 Trace Forms

Forms in TRACE are designed for clear, guided input across assets, scans, schedules, and reports. Validation, focused states, and structured fields help users complete security workflows quickly and accurately.

Generate Report

Evidence Source

vault-scan.vercel.app

Output Format

Executive PDF

Professional PDF document

Tactical JSON

Raw security metadata

Interactive HTML

Web-based analysis portal

Cancel

Reset

Generate Report

Figure 51 : Generate Report Form

Invite Team Member

Email Address

salemsyam231@gmail.com

Role

EDITOR

Send Invitation

Figure 52 : Inviter Member Form

Initiate Security Scan

Target Asset

Select Asset...

Scan Type

Quick Scan

Deep Scan

Authentication & Headers

Auto

Credentials

Cookies

None

Cancel

Reset

Start Scan

Figure 47: Initiate Scan Form

New Automated Schedule

Target Asset

Select target asset...

Scan Type

Quick Scan

Deep Scan

Frequency

WEEKLY

Execution Time

02:00 AM

Cancel

Reset

Deploy Schedule

Figure 48 : Add Schedule Form

Generate Report

Evidence Source

vault-scan.vercel.app

Output Format

Executive PDF

Professional PDF document

Tactical JSON

Raw security metadata

Interactive HTML

Web-based analysis portal

Cancel

Reset

Generate Report

Figure 54: Generate Report Form

Invite Team Member

Email Address

salemsyam231@gmail.com

Role

EDITOR

Send Invitation

Figure 53 : Invite Team Member Form

66

4.2 Frontend Architecture

Technologies: Next.js 15 (App Router), React 19, TypeScript, TanStack Query, TanStack Table, shadcn/ui, and Tailwind CSS v4.

4.2.1 Application Structure

The frontend uses Next.js App Router with route groups for clean separation:

- (auth)/ — Login and registration pages with unauthenticated layout.
- (dashboard)/ — Protected routes: overview, assets, scans, findings, reports, schedules, notifications, settings.
- components/ — Reusable UI components (dataTable, layout, ui).
- Features/ — Feature-specific components organized by domain (ai, assets, authentication, dashboard, scans, etc.).
- Services/ — API service layer for backend communication.
- Contexts/ — React context providers for authentication and theme state.
- hooks/ — Custom React hooks for SSE connections and data fetching.

4.2.2 State Management & Security

TanStack Query manages all asynchronous API communication with automatic caching, background refetching, and optimistic updates. Authentication uses JWT tokens stored securely with automatic refresh token rotation. The UI dynamically adapts visibility and actions based on user roles (Admin/Editor/Viewer).

4.3 Backend Architecture

The backend is built with NestJS and TypeScript, structured into feature-focused modules for authentication, organizations, assets, scans, findings, reports, schedules, notifications, dashboard analytics, and AI services. This modular architecture keeps the platform scalable, maintainable, and ready for future security workflows.

4.3.1 Authentication System

TRACE uses JWT-based authentication with Passport.js. Figure 30 shows the JWT strategy that extracts and validates Bearer tokens from request headers.



```
@Injectable()
export class JwtStrategy extends PassportStrategy(Strategy, 'jwt') {
  constructor(configService: ConfigService) {
    const secret = configService.get<string>('JWT_SECRET');
    if (!secret) throw new Error('JWT_SECRET is not defined');
    super({
      jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
      ignoreExpiration: false,
      secretOrKey: secret,
    });
  }
  validate(payload: JwtPayload): { userId: string; email: string } {
    if (!payload.sub) throw new UnauthorizedException('Invalid token');
    return { userId: payload.sub, email: payload.email };
  }
}
```

Figure 55: JWT Authentication Strategy (Code)

Figure 31 shows the JWT Auth Guard that protects all routes by default, with a decorator-based escape hatch for public endpoints.



```
@Injectable()
export class JwtAuthGuard extends AuthGuard('jwt') {
  constructor(private reflector: Reflector) { super(); }
  canActivate(context: ExecutionContext) {
    const isPublic = this.reflector.getAllAndOverride<boolean>(
      IS_PUBLIC_KEY, [context.getHandler(), context.getClass()]
    );
    if (isPublic) return true;
    return super.canActivate(context);
  }
}
```

Figure 56 : JWT Auth Guard (Code)

4.3.2 API Endpoints

ID	Endpoint	Method(s)	Description
1	/api	GET	Health/root API response.
2	/api/auth/register	POST	Register a new user account.
3	/api/auth/login	POST	Authenticate and receive JWT tokens.
4	/api/auth/refresh	POST	Refresh access token using refresh token.
5	/api/auth/logout	POST	Log out the current user.
6	/api/auth/me	GET	Retrieve the authenticated user profile.
7	/api/auth/profile	PATCH	Update user profile information.
8	/api/auth/change-password	POST	Change the authenticated user password.
9	/api/organizations	GET, POST	List user organizations or create a new organization.
10	/api/organizations/switch	POST	Switch the active organization context.
11	/api/organizations/:orgId	PATCH, DELETE	Update or delete an organization.
12	/api/organizations/:orgId/members	GET, POST	List organization members or add a member.
13	/api/organizations/:orgId/members/:memberId	PATCH, DELETE	Update member role or remove a member.
14	/api/assets	GET, POST	List assets or create a new asset.

ID	Endpoint	Method(s)	Description
15	/api/assets/stats	GET	Retrieve asset statistics.
16	/api/assets/:id	GET, PATCH, DELETE	Retrieve, update, or delete an asset.
17	/api/assets/bulk	POST	Create multiple assets at once.
18	/api/scans	GET, POST	List scans or initiate a new scan.
19	/api/scans/bulk	POST	Initiate multiple scans at once.
20	/api/scans/:id	GET, DELETE	Retrieve scan details or delete a scan.
21	/api/scans/:id/cancel	DELETE	Cancel a running or pending scan.
22	/api/scans/:id/progress	GET (SSE)	Stream real-time scan progress.
23	/api/scans/:id/findings	GET	Retrieve findings for a scan.
24	/api/scans/:id/raw-output	GET	Retrieve raw scan engine output.
25	/api/scans/:scanId/ai-analysis	GET	Get AI analysis for a scan.
26	/api/scans/:scanId/ai-analysis/retry	POST	Retry failed AI analysis.
27	/api/scan-findings	GET, DELETE	List findings or delete multiple findings.
28	/api/scan-findings/:id	DELETE	Delete a single finding.
29	/api/ai/chat	POST	Send a message to the AI chat assistant.
30	/api/ai/insights	GET	Retrieve AI dashboard insights.

ID	Endpoint	Method(s)	Description
31	/api/ai/findings/:findingId/remediation	POST	Generate an AI remediation guide for a finding.
32	/api/reports	GET, POST	List reports or generate a new report.
33	/api/reports/download/:id	GET	Download a generated report file.
34	/api/scan-schedules	GET, POST	List scan schedules or create a schedule.
35	/api/scan-schedules/:id	PATCH, DELETE	Update or delete a scan schedule.
36	/api/notifications	GET	List user notifications.
37	/api/notifications/read-all	PATCH	Mark all notifications as read.
38	/api/notifications/:id/read	PATCH	Mark a notification as read.
39	/api/dashboard/stats	GET	Retrieve dashboard summary statistics.
40	/api/dashboard/vulnerabilities-by-severity	GET	Retrieve vulnerability counts by severity.
41	/api/dashboard/vulnerability-trends	GET	Retrieve vulnerability trend data.
42	/api/dashboard/scan-activity	GET	Retrieve scan activity metrics.
43	/api/dashboard/recent-activity	GET	Retrieve recent dashboard activity.
44	/api/dashboard/top-vulnerabilities	GET	Retrieve top vulnerability summaries.
45	/api/dashboard/security-score	GET	Retrieve organization security score.
46	/api/docs	GET	Open Swagger API documentation.

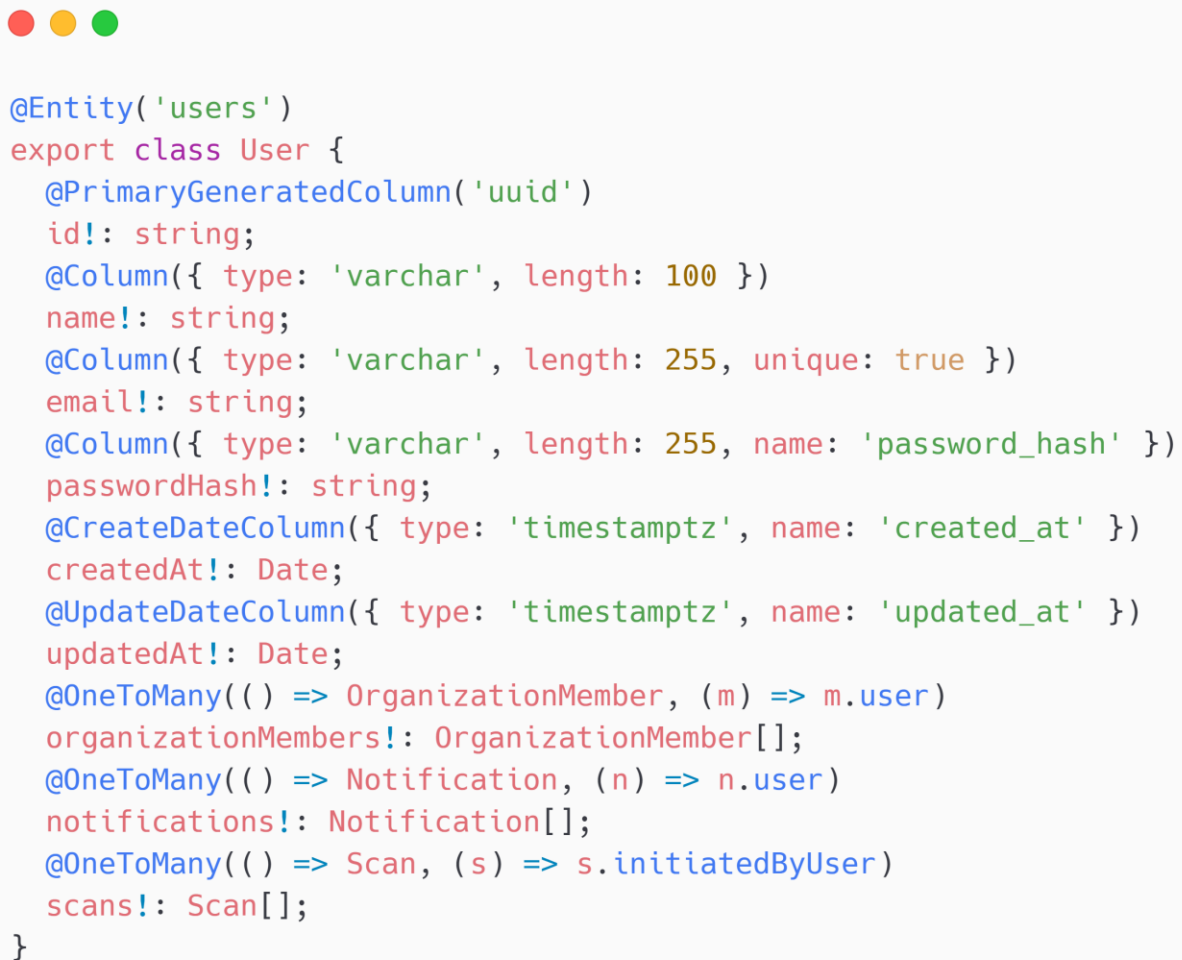
Table 5: API Endpoints

4.4 Database Layer

Technology: PostgreSQL 15, accessed via TypeORM with decorator-based entity definitions.

4.4.1 Entity Definitions

Figure 57 shows the User entity definition using TypeORM decorators:



```
@Entity('users')
export class User {
  @PrimaryGeneratedColumn('uuid')
  id!: string;
  @Column({ type: 'varchar', length: 100 })
  name!: string;
  @Column({ type: 'varchar', length: 255, unique: true })
  email!: string;
  @Column({ type: 'varchar', length: 255, name: 'password_hash' })
  passwordHash!: string;
  @CreateDateColumn({ type: 'timestampz', name: 'created_at' })
  createdAt!: Date;
  @UpdateDateColumn({ type: 'timestampz', name: 'updated_at' })
  updatedAt!: Date;
  @OneToMany(() => OrganizationMember, (m) => m.user)
  organizationMembers!: OrganizationMember[];
  @OneToMany(() => Notification, (n) => n.user)
  notifications!: Notification[];
  @OneToMany(() => Scan, (s) => s.initiatedByUser)
  scans!: Scan[];
}
```

Figure 57: User Entity — TypeORM Definition (Code)

Figure 58 shows the Scan entity with its relationships to Asset, Organization, ScanFinding, and AIAnalysis:

```

@Entity('scans')
export class Scan {
  @PrimaryGeneratedColumn('uuid')
  id!: string;
  @Column({ type: 'enum', enum: ScanType })
  type!: ScanType;
  @Column({ type: 'enum', enum: ScanStatus, default: ScanStatus.PENDING })
  status!: ScanStatus;
  @Column({ type: 'timestampz', name: 'started_at', nullable: true })
  startedAt!: Date | null;
  @Column({ type: 'timestampz', name: 'completed_at', nullable: true })
  completedAt!: Date | null;
  @ManyToOne(() => Asset, (a) => a.scans, { onDelete: 'CASCADE' })
  @JoinColumn({ name: 'asset_id' })
  asset!: Asset;
  @ManyToOne(() => Organization, (o) => o.scans, { onDelete: 'CASCADE' })
  @JoinColumn({ name: 'org_id' })
  organization!: Organization;
  @OneToMany(() => ScanFinding, (f) => f.scan)
  findings!: ScanFinding[];
  @OneToOne(() => AIAnalysis, (a) => a.scan)
  aiAnalysis!: AIAnalysis;
}
```

Figure 58 : Scan Entity — TypeORM Definition (Code)

4.4.2 Server Configuration

Component	Technology	Configuration
Database	PostgreSQL 15	Docker container (port 5432)
Cache/Queue	Redis 7	Docker container (port 6379)
Backend	NestJS (Node.js 20+)	Port 3001
Frontend	Next.js 15	Port 3000
Python Runtime	Python 3.11+	Used by scan scripts

Table 6 : Server Configuration

4.5 Scan Engine Layer

The Scan Engine is the core security testing component of TRACE. It is built on a modular micro-script architecture where each vulnerability check is implemented as an independent Python script. The engine currently includes 43 specialized scripts.

4.5.1 Script Runner Architecture

The ScriptRunnerService spawns child Python processes using a --target argument under a strict "Runner Contract."

```
@Injectable()
export class ScriptRunnerService {
  private readonly activePids = new Map<string, Set<number>>>();
  private readonly controllers = new Map<string, AbortController>();

  getSignal(scanId: string): AbortSignal {
    let ctrl = this.controllers.get(scanId);
    if (!ctrl) { ctrl = new AbortController(); this.controllers.set(scanId, ctrl); }
    return ctrl.signal;
  }

  async runScript(scriptName: string, args: string[], scanId?: string,
    timeout: number = 120000): Promise<ScriptResult[]> {
    const signal = scanId ? this.controllers.get(scanId)?.signal : undefined;
    if (signal?.aborted) throw new ScanCancelledError(scanId);

    const scriptPromise = this._executeScript(scriptName, args, scanId, signal);
    const abortPromise = new Promise<ScriptResult[]>((_ , reject) => {
      signal?.addEventListener('abort',
        () => reject(new ScanCancelledError(scanId)), { once: true });
    });

    return Promise.race([
      scriptPromise, abortPromise,
      new Promise<ScriptResult[]>((_ , reject) =>
        setTimeout(() => reject(new Error(`${scriptName} timed out`)), timeout))
    ]);
  }

  killScan(scanId: string): void {
    const ctrl = this.controllers.get(scanId);
    if (ctrl && !ctrl.signal.aborted) ctrl.abort();
    this.controllers.delete(scanId);
    const pids = this.activePids.get(scanId);
    if (pids) {
      for (const pid of pids) { try { process.kill(pid, 'SIGTERM'); } catch {} }
      this.activePids.delete(scanId);
    }
  }
}
```

Figure 59 : ScriptRunnerService — Python Process Spawning (Code)

4.5.2 Scan Orchestrator

The ScanOrchestratorService coordinates scanning in phases using Promise.allSettled for fault isolation. Figure 35 shows the Quick Scan flow:



```
async runQuickScan(jobData: ScanJobData): Promise<void> {
  const { scanId, assetValue, assetType } = jobData;
  const signal = this.scriptRunner.getSignal(scanId);
  await this.updateScanStatus(scanId, ScanStatus.RUNNING);
  this.emitProgress(scanId, 0, 'Starting quick scan');
  const allResults: ScriptFinding[][] = [];

  // Phase 0: Auth Discovery (0-10%)
  this.emitProgress(scanId, 2, 'Attempting authentication bypass...');
  const authResults = await Promise.allSettled([
    this.scriptRunner.runScript('chk_00_auth_discovery.py',
      ['--target', assetValue], scanId, 40000)
  ]);
  allResults.push(...this.extractSettledResults(authResults, scanId));

  // Phase 1: Infrastructure (10-30%)
  this.throwIfCancelled(signal, scanId);
  this.emitProgress(scanId, 12, 'Scanning infrastructure...');
  const phase1 = await Promise.allSettled([
    this.scriptRunner.runScript('chk_quick.py', ['--target', assetValue], scanId),
    this.scriptRunner.runScript('ssl_checker.py', ['--target', assetValue], scanId),
    this.scriptRunner.runScript('chk_22_cookie_security.py', ['--target', assetValue], scanId),
  ]);
  allResults.push(...this.extractSettledResults(phase1, scanId));

  // Phase 2: Vulnerability Testing (30-70%)
  this.throwIfCancelled(signal, scanId);
  const phase2 = await Promise.allSettled([
    this.scriptRunner.runScript('chk_01_sql.py', ['--target', assetValue], scanId),
    this.scriptRunner.runScript('chk_02_xss.py', ['--target', assetValue], scanId),
    this.scriptRunner.runScript('chk_03_csrf.py', ['--target', assetValue], scanId),
  ]);
  allResults.push(...this.extractSettledResults(phase2, scanId));

  // Phase 3: Aggregate & Save (90-100%)
  const aggregated = this.aggregator.aggregate(allResults);
  await this.saveFindings(scanId, aggregated);
  await this.updateScanStatus(scanId, ScanStatus.COMPLETED);
  this.eventEmitter.emit('scan.completed', { scanId, orgId: jobData.orgId });
}
```

Figure 60 : ScanOrchestratorService — Phase-Based Execution (Code)

4.5.3 Scan Phases

Phase	Name	Scripts	Description
Phase 0	Auth Discovery	chk_00_auth_discovery.py	Login form discovery and authentication bypass attempts.
Phase 1	Infrastructure	chk_17_nmap.py, service_fingerprint.py, ssl_checker.py	Port scanning, service detection, SSL/TLS analysis.
Phase 1.5	DNS Recon	chk_09_subdomain.py, chk_10_dns.py	Subdomain enumeration and DNS configuration analysis.
Phase 2a	Injection	chk_01_sql.py, chk_02_xss.py, chk_06_cmdi.py, chk_12_ssti.py	SQL injection, XSS, command injection, SSTI testing.
Phase 2b	Access Control	chk_03_csrf.py, chk_05_lfi.py, chk_14_idor.py, chk_13_ssrf.py	CSRF, LFI, IDOR, and SSRF testing.
Phase 2c	Protocol	20+ scripts (CORS, JWT, CRLF, broken auth, etc.)	Protocol-level vulnerability testing.
Phase 3	Configuration	chk_07_misconfig.py, chk_08_sensitive_data.py	Security misconfiguration and data exposure checks.

Table 7 : Scan Phases and Scripts

4.5.4 Quick Scan vs Deep Scan

Aspect	Quick Scan	Deep Scan
Scripts Count	~10 scripts	43+ scripts
Timeout per Script	30 seconds	Up to 4 minutes
Coverage	SQLi, XSS, CSRF, headers, SSL, WAF	Full: NoSQL, LDAP, deserialization, cloud, subdomain takeover, etc.
Typical Duration	1–2 minutes	3–5 minutes
Use Case	Quick security check	Comprehensive assessment

Table 8 : Quick Scan vs Deep Scan Comparison

4.5.5 Aggregator Service

The AggregatorService normalizes severity levels and deduplicates findings:



```
@Injectable()
export class AggregatorService {
  aggregate(scriptResults: ScriptFinding[][]): AggregatedFinding[] {
    const allFindings = scriptResults.flat();
    const aggregated = allFindings.map((f) => ({
      vulnerabilityName: f.vulnerability || 'Unknown',
      severity: this.normalizeSeverity(f.severity),
      location: f.location || 'unknown',
      evidence: f.evidence || '',
      category: f.category || 'GENERAL',
      cveId: f.cve_id || null,
      rawOutput: JSON.stringify(f.raw_details || {}),
    }));
    // Deduplicate by name + location
    const seen = new Set<string>();
    const deduped: AggregatedFinding[] = [];
    for (const f of aggregated) {
      const key = `${f.vulnerabilityName}::${f.location}`;
      if (!seen.has(key)) { seen.add(key); deduped.push(f); }
    }
    return deduped;
  }
  normalizeSeverity(s: string): SeverityLevel {
    const map = { CRITICAL: SeverityLevel.CRITICAL, HIGH: SeverityLevel.HIGH,
      MEDIUM: SeverityLevel.MEDIUM, LOW: SeverityLevel.LOW };
    return map[s?.toUpperCase()] || SeverityLevel.LOW;
  }
}
```

Figure 61 : AggregatorService — Finding Normalization (Code)

4.6 AI Analysis Layer

TRACE integrates Google Gemini AI for intelligent scan analysis, automatically triggered when a scan completes via the event-driven architecture.

4.6.1 Automated Scan Analysis

Figure 62 shows the AiAnalysisService which listens for scan.completed events:



```
@Injectable()
export class AiAnalysisService {
  @OnEvent('scan.completed')
  handleScanCompleted(payload: { scanId: string }): void {
    this.analyzeScan(payload.scanId).catch((err) =>
      this.logger.error(`Background AI analysis error: ${err.message}`));
  }

  async analyzeScan(scanId: string): Promise<void> {
    const analysis = this.analysisRepo.create({
      scanId, status: AIAalysisStatus.PROCESSING, geminiModel });
    const saved = await this.analysisRepo.save(analysis);
    try {
      const scan = await this.loadScanWithContext(scanId);
      const systemPrompt = this.promptBuilder.buildSystemPrompt();
      const userPrompt = this.promptBuilder.buildAnalysisPrompt(
        scan, scan.asset, scan.findings);
      const result = await this.callGemini(geminiModel, systemPrompt, userPrompt);
      const parsed = this.parseGeminiResponse(result.text);
      saved.status = AIAalysisStatus.COMPLETED;
      saved.riskScore = parsed.riskScore;
      saved.riskLevel = this.mapRiskLevel(parsed.riskLevel);
      saved.analysisText = parsed.executiveSummary;
      saved.recommendations = parsed.recommendations;
      saved.keyFindings = parsed.keyFindings;
      await this.analysisRepo.save(saved);
      this.eventEmitter.emit('ai.analysis.completed', { scanId });
    } catch (error) {
      saved.status = AIAalysisStatus.FAILED;
      saved.errorMessage = error.message;
      await this.analysisRepo.save(saved);
    }
  }
}
```

Figure 62 : AI Analysis Service — Event-Driven Trigger (Code)

4.6.2 Prompt Engineering

Figure 63 shows how the PromptBuilderService constructs structured prompts with scan context:



```
@Injectable()
export class PromptBuilderService {
  buildSystemPrompt(): string {
    return `You are an expert cybersecurity analyst with deep knowledge
    of OWASP Top 10, CVE database, and penetration testing. Analyze
    vulnerability scan results and provide a comprehensive, actionable
    security assessment. Always respond with valid JSON only.`;
  }

  buildAnalysisPrompt(scan: Scan, asset: Asset, findings: ScanFinding[]) {
    const findingsList = findings.map((f) =>
      `- [${f.vulnerability.severity}] ${f.vulnerability.name}: ` +
      `${f.evidence} at ${f.location}`).join('\n');
    return `Analyze this vulnerability scan report...
    ASSET: ${asset.name} (${asset.type}) - ${asset.value}
    SCAN TYPE: ${scan.type} | FINDINGS: ${findings.length}
    ${findingsList || 'No vulnerabilities detected.'}
    Respond with JSON: { executiveSummary, riskScore (0-100),
      riskLevel, keyFindings[], recommendations[], technicalDetails,
      attackVectors[], complianceNotes }`;
  }
}
```

Figure 63 : Prompt Builder Service — Gemini Prompt Construction (Code)

4.7 Report Generation

Format	Technology	Description
PDF	Puppeteer (Headless Chrome)	Professional reports with severity badges, findings tables, and AI analysis.
HTML	Template Engine	Self-contained HTML files with inline CSS.
JSON	Native	Machine-readable structured data for integration with other tools.

Table 9 : Report Generation Formats

4.8 Development Tools and Requirements

Tool / Technology	Purpose
Visual Studio Code	Primary code editor for TypeScript and Python development.
Node.js 20+	JavaScript runtime for NestJS backend and Next.js frontend.
Python 3.11+	Runtime for the 43 vulnerability scanning scripts.
Docker & Docker Compose	Containerization of PostgreSQL, Redis, and application services.
PostgreSQL 15	Relational database for all persistent data.
Redis 7	In-memory store for BullMQ job queue management.
Git & GitHub	Version control and collaborative development.
Postman	API testing and endpoint validation.
Puppeteer	Headless Chrome for PDF report generation.
Google Gemini API	AI-powered vulnerability analysis.

Table 10 : Development Tools and Technologies

4.9 Testing and Evaluations

4.9.1 Testing Methodology

The system was tested using a combination of manual testing and functional testing approaches. Each module was tested individually to verify correct behavior in isolation, followed by integration testing to ensure all components work together seamlessly.

Testing covered the following areas:

- Authentication: Registration, login, token refresh, and role-based access control.
- Asset Management: CRUD operations with format validation for each asset type.
- Scan Execution: Quick and deep scan workflows, progress tracking, and cancellation.
- Vulnerability Detection: Accuracy of scanning scripts against known vulnerable targets.
- AI Analysis: Gemini API integration, response parsing, and error handling.
- Report Generation: PDF, HTML, and JSON output correctness.
- Scheduling: Recurring scan creation, execution, and management.
- Notifications: Event-triggered alerts and read status management.
- Role-Based Access: Permission enforcement across all API endpoints.

4.9.2 Test Cases

Table 11 presents the key test cases executed to validate system functionality.

No.	Module	Test Case	Expected Result	Status
1	Auth	Register with valid email and password.	User created, JWT tokens returned, workspace created.	Pass
2	Auth	Login with correct credentials.	JWT tokens returned with user data.	Pass
3	Auth	Login with incorrect password.	401 Unauthorized error.	Pass
4	Auth	Register with existing email.	409 Conflict error.	Pass
5	Assets	Add a new domain asset.	Asset created and listed.	Pass
6	Assets	Add asset with invalid type.	Validation error returned.	Pass
7	Assets	Delete an existing asset.	Asset removed from database.	Pass
8	Scans	Initiate a Quick Scan.	Scan created with PENDING status, job dispatched to BullMQ.	Pass
9	Scans	Initiate a Deep Scan.	All 43+ scripts executed, findings aggregated.	Pass
10	Scans	Cancel a running scan.	All active scripts terminated, status set to CANCELLED.	Pass

11	Scans	Monitor scan progress via SSE.	Real-time percentage and phase messages received.	Pass
12	Findings	View findings for completed scan.	Findings displayed with severity, category, evidence.	Pass
13	Findings	Filter findings by severity.	Only matching severity findings shown.	Pass
14	AI	Auto-trigger AI analysis on scan complete.	Analysis record created with risk score and recommendations.	Pass
15	AI	Retry failed AI analysis.	New analysis generated successfully.	Pass
16	AI	Send message to AI Chat.	Context-aware response received.	Pass
17	Reports	Generate PDF report.	PDF file created and downloadable.	Pass
18	Reports	Generate JSON report.	Valid JSON file with scan data.	Pass
19	Schedule	Create weekly scan schedule.	Schedule saved with next execution time.	Pass
20	Notifications	Receive scan completion notification.	Notification created and displayed.	Pass
21	RBAC	Viewer attempts to initiate scan.	403 Forbidden error returned.	Pass
22	RBAC	Editor adds an asset.	Asset created successfully.	Pass

23	RBAC	Editor attempts to manage members.	403 Forbidden error returned.	Pass
----	------	------------------------------------	-------------------------------	------

Table 11 : Test Cases and Results

4.9.3 Limitations

The following limitations were identified during testing:

1. **Authenticated Scanning Limitations:** Authenticated scanning requires manually provided session cookies. Complex multi-step login flows with CAPTCHA, multi-factor authentication, or JavaScript-heavy login forms are not automated. Users must extract session cookies manually and provide them as scan parameters.
2. **Black-Box Testing Only:** All scanning scripts perform black-box testing by sending HTTP requests and analyzing responses. The system cannot analyze application source code (static analysis) or perform white-box testing with access to internal application state.
3. **Scan Duration Variability:** Deep Scan duration varies significantly depending on target response time, network latency, and the number of discoverable endpoints. Targets with slow response times or rate limiting may cause individual scripts to time out.
4. **AI Analysis Dependency:** AI analysis requires an active internet connection and a valid Google Gemini API key. If the Gemini API is unavailable or rate-limited, analysis will fail (though it can be retried). Analysis quality depends on the AI model's current capabilities.
5. **Scope Limitation:** The system targets web applications and network infrastructure. Mobile application testing (iOS/Android), IoT device scanning, and physical security assessments are not supported.

4.9.4 System Evaluation Metric

To assess the practical value of TRACE, the platform's performance was evaluated against traditional, manual security assessment workflows. Typically, running a manual infrastructure check via command-line tools followed by isolated application security script testing takes a security engineer roughly 20 to 30 minutes of environment configuration, manual execution, and disjointed data translation.

TRACE completely automates this lifecycle. By leveraging concurrent process orchestration via BullMQ and fault isolation using Promise.allSettled patterns, the platform manages to complete a comprehensive Deep Scan across both network and web application layers, normalize the findings, and generate a tailored AI-remediation roadmap within less than 3 minutes. This represents a significant optimization in time efficiency and eliminates manual context-switching errors for system administrators.

Key performance metrics observed during testing:

- Quick Scan average completion time: 45–90 seconds.
- Deep Scan average completion time: 2–4 minutes.
- AI Analysis generation time: 10–30 seconds after scan completion.
- PDF Report generation time: 3–5 seconds.
- Concurrent scan support: Multiple scans can run simultaneously through BullMQ job queue management.

Chapter 5

Conclusion and Future Work

CHAPTER 5: CONCLUSION AND FUTURE WORK

5.1 Conclusion

This project successfully developed an automated vulnerability scanning platform that integrates network and web application security testing into a unified system. TRACE addresses the challenges of tool fragmentation, manual analysis overhead, inconsistent reporting, and lack of continuous monitoring identified in the problem definition.

The system achieves its primary objectives through:

- A modular scan engine with 43 Python scripts covering vulnerabilities from SQL injection and cross-site scripting to subdomain takeover, cloud storage exposure, and NoSQL injection.
- Two scan modes (Quick and Deep) providing flexible coverage based on time constraints and security requirements.
- AI-powered analysis using Google Gemini for automated risk scoring (0–100), executive summaries, key findings with impact assessment, and prioritized, actionable remediation recommendations.
- Real-time scan monitoring using Server-Sent Events (SSE) for live progress tracking with phase-level status messages.
- A multi-tenant platform with role-based access control (Admin, Editor, Viewer) enabling team-based security management.
- Automated report generation in PDF, HTML, and JSON formats for flexible security documentation and compliance reporting.
- Recurring scan scheduling (Daily, Weekly, Monthly) for continuous security monitoring without manual intervention.
- An interactive AI Chat assistant with context-aware security guidance based on actual scan data.

The platform demonstrated significant efficiency gains during evaluation, completing comprehensive deep scans in under 3 minutes compared to 20–30 minutes for equivalent manual workflows. The fault-tolerant architecture ensures that individual script failures do not

compromise the overall scan, while the event-driven AI analysis pipeline provides immediate, intelligent insights upon scan completion.

TRACE transforms vulnerability scanning from a manual, fragmented process into a continuous, centralized security management platform accessible to both technical and non-technical users. The modular architecture ensures that new vulnerability checks can be added by simply creating a new Python script that follows the established Runner Contract, without modifying the core system.

5.2 Future Work

The following enhancements are planned for future development of TRACE:

5.2.1 Mobile Application

A companion mobile application for iOS and Android would allow security teams to monitor scan progress, receive push notifications for critical vulnerabilities, and view scan results on the go. The mobile app would communicate with the existing NestJS backend via the REST API, requiring minimal backend changes. Key features would include scan initiation, real-time progress monitoring, finding review, and configurable alert thresholds for severity-based push notifications.

5.2.2 CI/CD Integration

Integration plugins for popular CI/CD platforms — Jenkins, GitHub Actions, and GitLab CI — would enable security scanning as part of the software development lifecycle. Developers could trigger TRACE assessments automatically on code deployments, with configurable quality gates that prevent deployment if critical vulnerabilities are detected. This shift-left security approach would catch vulnerabilities earlier in the development process.

5.2.3 CVE Database Integration

Automatic cross-referencing of detected vulnerabilities with the Common Vulnerabilities and Exposures (CVE) database would enrich findings with official vulnerability identifiers, CVSS

scores, and links to detailed advisories. This integration would provide additional context for remediation prioritization and support compliance reporting requirements.

5.2.4 Agent-Based Internal Scanning

Deploying lightweight scanning agents on internal network hosts would extend TRACE's reach beyond externally accessible assets. These agents would perform local vulnerability assessments and report findings back to the central platform, enabling comprehensive internal network security monitoring without requiring inbound network access.

5.2.5 ML-Based False Positive Reduction

Training machine learning models on historical scan data would improve detection accuracy by learning to distinguish true positives from false alarms. Over time, the system would build a knowledge base of confirmed and rejected findings, using this data to assign confidence scores to new detections and reduce the manual verification burden on security teams.

5.2.6 Advanced Authentication Handling

Implementing automated login handling for complex multi-factor authentication flows would enable TRACE to scan authenticated areas of web applications without manual cookie extraction. This could involve headless browser automation with Puppeteer to navigate login forms, handle CAPTCHA challenges, and manage session tokens automatically.

5.2.7 Compliance Framework Mapping

Mapping detected vulnerabilities to established compliance frameworks — PCI DSS, HIPAA, ISO 27001, and SOC 2 — would provide organizations with compliance-ready reports. Each finding would be tagged with relevant framework requirements, helping security teams demonstrate compliance status and identify gaps in their security controls.

REFERENCES

- [1] OWASP Foundation, "OWASP Top Ten Web Application Security Risks," 2021. [Online]. Available: <https://owasp.org/www-project-top-ten/>
- [2] Gordon Lyon (Fyodor Vaskovich), "Nmap Network Scanning," Nmap.org, 2023. [Online]. Available: <https://nmap.org/book/>
- [3] OWASP Foundation, "OWASP ZAP – Zed Attack Proxy," 2024. [Online]. Available: <https://www.zaproxy.org/>
- [4] NestJS, "NestJS – A progressive Node.js framework," 2024. [Online]. Available: <https://docs.nestjs.com/>
- [5] Google, "Gemini API Documentation," 2024. [Online]. Available: <https://ai.google.dev/docs>
- [6] BullMQ, "BullMQ – Premium Message Queue for Node.js," 2024. [Online]. Available: <https://docs.bullmq.io/>
- [7] FIRST, "Common Vulnerability Scoring System v3.1," 2019. [Online]. Available: <https://www.first.org/cvss/>
- [8] Vercel, "Next.js Documentation," 2024. [Online]. Available: <https://nextjs.org/docs>
- [9] TypeORM, "TypeORM – ORM for TypeScript and JavaScript," 2024. [Online]. Available: <https://typeorm.io/>
- [10] Google, "Puppeteer Documentation," 2024. [Online]. Available: <https://pptr.dev/>
- [11] PostgreSQL, "PostgreSQL 15 Documentation," 2024. [Online]. Available: <https://www.postgresql.org/docs/15/>
- [12] Redis Ltd., "Redis 7 Documentation," 2024. [Online]. Available: <https://redis.io/docs/>
- [13] Tenable, "Nessus Vulnerability Scanner Documentation," 2024. [Online]. Available: <https://docs.tenable.com/nessus/>
- [14] PortSwigger, "Burp Suite Documentation," 2024. [Online]. Available: <https://portswigger.net/burp/documentation>

- [15] Python Software Foundation, "Requests: HTTP for Humans," 2024. [Online]. Available: <https://docs.python-requests.org/>
- [16] Docker Inc., "Docker Documentation," 2024. [Online]. Available: <https://docs.docker.com/>
- [17] Tailwind Labs, "Tailwind CSS Documentation," 2024. [Online]. Available: <https://tailwindcss.com/docs>
- [18] shadcn, "shadcn/ui – Beautifully designed components," 2024. [Online]. Available: <https://ui.shadcn.com/>
- [19] TanStack, "TanStack Query Documentation," 2024. [Online]. Available: <https://tanstack.com/query/>
- [20] IETF, "RFC 7519 – JSON Web Token (JWT)," 2015. [Online]. Available: <https://tools.ietf.org/html/rfc7519>
- [21] Agile Alliance, "Agile Manifesto," 2001. [Online]. Available: <https://agilemanifesto.org/>

CONCLUSION

في ظل التحول الرقمي المتسارع، تواجه المؤسسات تحديات متزايدة في مجال الأمن السيبراني. أدوات فحص الثغرات التقليدية مجزأة ومعقدة وتتطلب خبرة تقنية عالية، مما يترك فجوات خطيرة في التغطية الأمنية .

منصة TRACE هي منصة آلية متكاملة لفحص الثغرات الأمنية تجمع بين فحص الشبكات وتطبيقات الويب في نظام موحد . وتتضمن المنصة محرك فحص يحتوي على 43 سكريبت بايثون متخصص يغطي مجموعة شاملة من فئات الثغرات الأمنية بما في ذلك حقن SQL وهجمات XSS وثغرات CSRF وغيرها .

يتميز النظام بدمج الذكاء الاصطناعي من Google Gemini لتحليل نتائج الفحص تلقائياً وتقديم تقييم شامل يتضمن درجة المخاطر والملخص التنفيذي والنتائج الرئيسية وتوصيات المعالجة ذات الأولوية .

تم بناء المنصة باستخدام تقنيات حديثة تشمل 15 Next.js للواجهة الأمامية و NestJS للواجهة الخلفية و PostgreSQL لقاعدة البيانات و Redis مع BullMQ لإدارة المهام في الخلفية .

أظهر النظام تحسناً كبيراً في الكفاءة حيث أكمل عمليات الفحص الشاملة في أقل من 3 دقائق مقارنة بـ 20-30 دقيقة للعمليات اليدوية المكافئة .

يجول TRACE فحص الثغرات من عملية يدوية مجزأة إلى منصة إدارة أمنية مركزية ومستمرة يمكن الوصول إليها من قبل المستخدمين التقنيين وغير التقنيين على حد سواء.

جامعة دمياط

كلية الحاسبات والذكاء الاصطناعي

مشروع التخرج

2025 / 2026

الماسح الآلي للثغرات الأمنية (TRACE)

إعداد

سالم محمد سالم محمد صيام

يوسف محمد ربيع عبد الوهاب

نور الإسلام حمدي مسعد إبراهيم عيد

مصطفى محمد السيد محمود رضوان

زياد محمد نجيب البدوي

محمد أحمد أبو الفتوح

تحت إشراف

د. أحمد محمد ربيع